CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN TẬP

CẨM NANG DSA

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT • 94 TRANG

```python
ids = list(range(1_000_000))
print(999_999 in ids)    # quét lần lượt

ids = set(ids)
print(999_999 in ids)    # nhảy thẳng tới nơi
```

Cùng đáp án, chênh hơn 1.000 lần. Vì sao?

Big-O | Cây | Đồ thị | DP | Phỏng vấn

Bản đồ cẩm nang

Mới học thì đi theo thứ tự. Mọi đoạn code đều đã chạy thật.

1 NỀN TẢNG VÀ BIG-O Trang 3

2 MẢNG VÀ CHUỖI Trang 11

3 LINKED LIST, STACK, QUEUE Trang 20

4 HASH TABLE Trang 28

5 CÂY VÀ HEAP Trang 32

6 TÌM KIẾM VÀ SẮP XẾP Trang 48

7 ĐỒ THỊ Trang 58

8 THIẾT KẾ GIẢI THUẬT Trang 69

9 PHỎNG VẤN VÀ LỘ TRÌNH Trang 83

1 DSA là gì?

Cấu trúc dữ liệu là cách sắp xếp dữ liệu. Giải thuật là các bước xử lý dữ liệu đó.

Bài toán quen thuộc	Nguyên lý phía sau
Gợi ý khi gõ ô tìm kiếm	**Trie**
Tìm đường ngắn nhất	**Dijkstra, A***
Tra cứu người dùng theo id	**Hash table, B+Tree**
Nút Undo	**Stack**
Hàng đợi in, hàng đợi tin nhắn	**Queue**
Gợi ý bạn chung	**Duyệt đồ thị**
So sánh hai phiên bản file	**LCS, diff**

Hệ thống thật thường phức tạp hơn. Bảng trên chỉ cho thấy nguyên lý cốt lõi.

Học DSA khỏi lan man

1 **Vẽ tay trước**
Chạy thuật toán trên giấy với 5 phần tử trước khi code.

2 **Tự viết lại**
Gấp tài liệu lại, viết từ đầu. Đọc hiểu chưa phải là làm được.

3 **Nói được Big-O**
Mỗi lời giải đều nêu thời gian và bộ nhớ.

4 **Luyện theo dạng**
Làm 5 đến 10 bài cùng một kỹ thuật rồi mới đổi sang dạng khác.

5 **Ôn lại có lịch**
Làm lại bài cũ sau 3 ngày và sau 1 tuần.

Mục tiêu không phải thuộc lời giải, mà là nhìn đề và nhận ra dạng bài.

2 Big-O là gì?

Big-O cho biết số bước tăng thế nào khi dữ liệu tăng, không phải số giây.

```
import math

print(f"{'n':<11}{'log n':<7}{'n log n':<12}n^2")
for n in (10, 1_000, 1_000_000):
    lg = round(math.log2(n))
    print(f"{n:<11}{lg:<7}{n * lg:<12}{n * n}")
```

KẾT QUẢ KHI CHẠY

```
n           log n  n log n     n^2
10          3      30          100
1000        10     10000       1000000
1000000     20     20000000    1000000000000
```

Big-O	Tên gọi	Ví dụ
O(1)	Hằng số	Lấy `a[i]`
O(log n)	Logarit	Tìm kiếm nhị phân
O(n)	Tuyến tính	Duyệt mảng một lần
O(n log n)	Tuyến tính logarit	Merge sort
O(n²)	Bình phương	Hai vòng lặp lồng nhau
O(2ⁿ)	Lũy thừa	Thử mọi tập con
O(n!)	Giai thừa	Thử mọi hoán vị

Bỏ hằng số: O(2n) là O(n). Giữ số hạng lớn nhất: O(n² + n) là O(n²).

3 Đọc Big-O từ code

Đếm xem mỗi dòng chạy bao nhiêu lần khi n tăng.

```
def examples(a):
    n = len(a)
    x = a[0]                    # O(1)
    for v in a:                 # O(n)
        x += v
    for i in range(n):          # O(n²): hai vòng lồng nhau
        for j in range(n):
            x += 1
    i = n
    while i > 1:                # O(log n): mỗi lần chia đôi
        i //= 2
    return x                    # tổng cộng: O(n²)

print(examples([1, 2, 3, 4]))
```

KẾT QUẢ KHI CHẠY

27

Mẫu code	Độ phức tạp
Hai vòng nối tiếp, cỡ n và m	**O(n + m)**
Hai vòng lồng nhau, cỡ n và m	**O(n · m)**
`i *= 2` hoặc `i //= 2` trong vòng lặp	**O(log n)**
Vòng ngoài n, bên trong chia đôi	**O(n log n)**
Đệ quy gọi 2 nhánh, sâu n tầng	**O(2ⁿ)**

4 Bộ nhớ, trường hợp xấu nhất và khấu hao

Một lời giải tốt được đánh giá cả về thời gian lẫn bộ nhớ dùng thêm.

```
def total(a):                  # bộ nhớ thêm O(1)
    s = 0
    for v in a:
        s += v
    return s

def doubled(a):                # bộ nhớ thêm O(n)
    return [v * 2 for v in a]

def depth(n):                  # O(n): n khung gọi hàm
    return 0 if n == 0 else 1 + depth(n - 1)

print(total([1, 2, 3]), doubled([1, 2]), depth(5))
```

KẾT QUẢ KHI CHẠY

```
6 [2, 4] 5
```

Khái niệm	Ý nghĩa
Tốt nhất	Đầu vào thuận lợi nhất, ít dùng để đánh giá
Trung bình	Kỳ vọng trên đầu vào thường gặp
Xấu nhất	Mức cam kết, phỏng vấn hay hỏi nhất
Khấu hao	Trung bình trên một chuỗi thao tác, như `append`

Nhìn giới hạn n, đoán lời giải

Giới hạn trong đề là gợi ý lớn nhất về độ phức tạp cần đạt.

Giới hạn n	Cần đạt	Kỹ thuật hay gặp
n ≤ 10	O(n!)	Hoán vị, quay lui
n ≤ 20	$O(2^n)$	Quay lui, bitmask
n ≤ 500	$O(n^3)$	Floyd-Warshall, DP
n ≤ 5.000	$O(n^2)$	DP hai chiều
n ≤ 10^6	O(n log n)	Sắp xếp, heap, tìm nhị phân
n ≤ 10^8	O(n)	Two pointers, hash
n ≤ 10^{18}	O(log n)	Tìm nhị phân, lũy thừa nhanh

Ước lượng nhanh: C++ xử lý khoảng 10^8 phép tính đơn giản mỗi giây, Python chậm hơn nhiều.

Đây là kinh nghiệm để định hướng, không phải quy tắc tuyệt đối.

5 Đệ quy và call stack

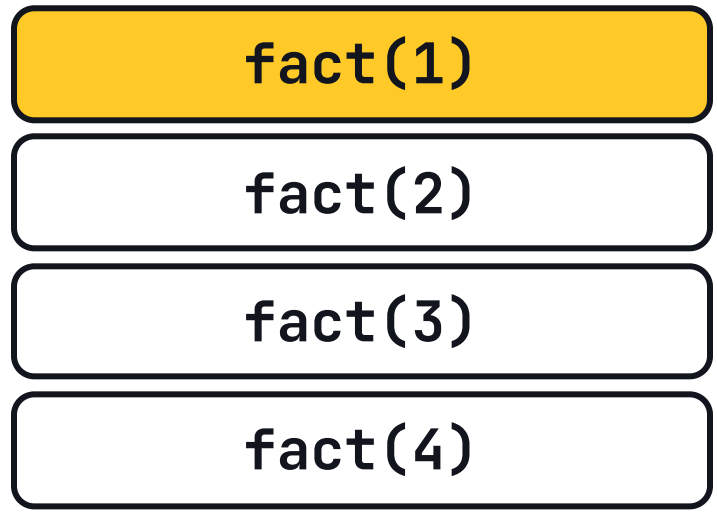
Đệ quy là hàm gọi lại chính nó trên bài toán nhỏ hơn, cho tới khi gặp điều kiện dừng.

```
def fact(n):
    if n <= 1:                  # điều kiện dừng
        return 1
    return n * fact(n - 1)      # thu nhỏ bài toán

print(fact(4))
```

KẾT QUẢ KHI CHẠY

```
24
```

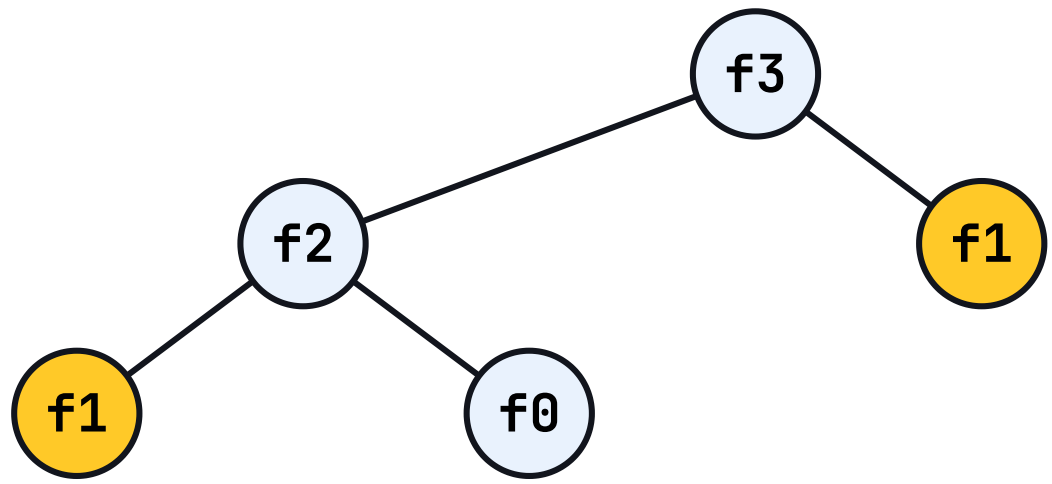


Call stack khi chạy tới đáy. fact(1) trả về trước, rồi lần lượt quay ngược lên.

- Mỗi lời gọi thêm một khung trên call stack, nên tốn bộ nhớ O(độ sâu).
- Thiếu điều kiện dừng sẽ gặp `RecursionError`. Python mặc định giới hạn khoảng 1.000 tầng.
- Viết đệ quy theo 3 câu: khi nào dừng, thu nhỏ thế nào, ghép kết quả ra sao.

6 Bẫy đệ quy: tính đi tính lại

Fibonacci đệ quy tính lại cùng một giá trị rất nhiều lần. Ghi nhớ giúp mỗi giá trị chỉ tính một lần.




```
from functools import cache

calls = 0
def fib(n):
    global calls
    calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(20), calls)      # O(2ⁿ)

calls = 0
@cache
def fib2(n):
    global calls
    calls += 1
    return n if n < 2 else fib2(n - 1) + fib2(n - 2)

print(fib2(20), calls)     # O(n)
```

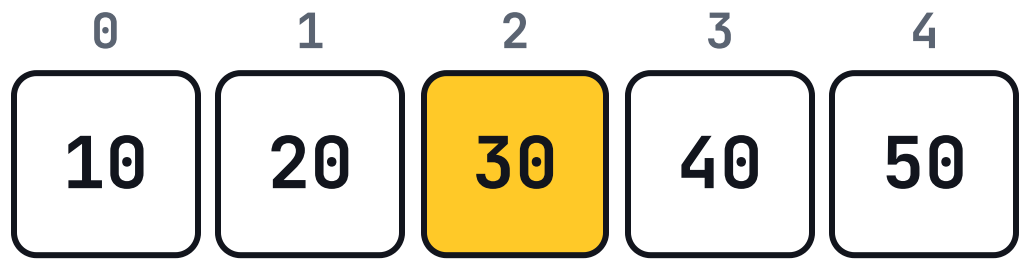
KẾT QUẢ KHI CHẠY

```
6765 21891
6765 21
```

Đây chính là ý tưởng mở đầu của Quy hoạch động ở Phần 8.

7 Mảng (Array)

Các phần tử nằm liền nhau trong bộ nhớ, nên biết chỉ số là đi thẳng tới được.




```
a = [10, 20, 30, 40, 50]
print(a[2])          # O(1): tới thẳng chỉ số 2
a.insert(1, 15)      # O(n): dời các phần tử phía sau
print(a)
a.pop(0)             # O(n): dồn cả mảng lên
print(a)
a.append(60)         # O(1) khấu hao
print(a, len(a))
```

KẾT QUẢ KHI CHẠY

```
30
[10, 15, 20, 30, 40, 50]
[15, 20, 30, 40, 50]
[15, 20, 30, 40, 50, 60] 6
```

MẠNH

Truy cập theo chỉ số rất nhanh, dữ liệu liền nhau nên thân thiện với bộ nhớ đệm.

YẾU

Chèn và xóa ở đầu hoặc giữa phải dời nhiều phần tử.

8 Mảng động lớn lên thế nào?

Khi đầy, mảng động xin vùng lớn hơn rồi chép dữ liệu sang. Nhờ tăng theo tỉ lệ, `append` vẫn O(1) khấu hao.

```
class DynArray:
    def __init__(self):
        self.cap, self.size, self.copies = 1, 0, 0
        self.data = [None]

    def append(self, x):
        if self.size == self.cap:       # đầy thì gấp đôi
            self.cap *= 2
            self.data += [None] * self.size
            self.copies += self.size
        self.data[self.size] = x
        self.size += 1

d = DynArray()
for i in range(1000):
    d.append(i)
print(d.cap, d.copies)
```

KẾT QUẢ KHI CHẠY

```
1024 1023
```

- 1.000 lần thêm chỉ tốn 1.023 lần chép, trung bình khoảng 1 lần chép mỗi phần tử.
- Nếu mỗi lần chỉ tăng thêm 1 ô, tổng chi phí sẽ là $O(n^2)$.
- Python list tăng theo tỉ lệ nhỏ hơn gấp đôi, nhưng nguyên lý giống nhau.

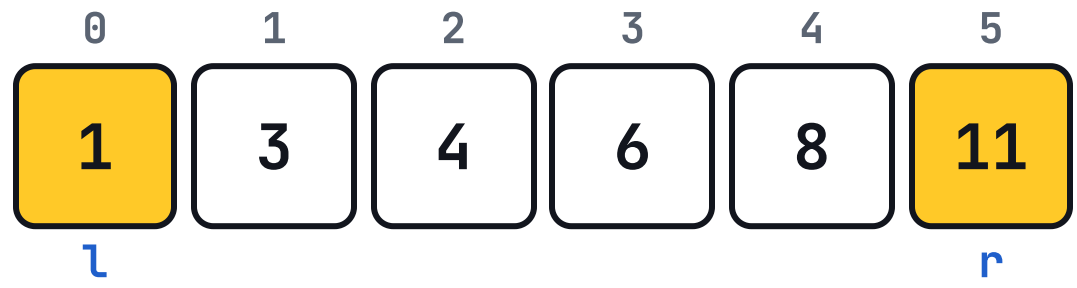
Chi phí của list trong Python

Thao tác	Chi phí	Ghi chú
`a[i]`, `a[i] = x`	**O(1)**	Truy cập trực tiếp
`a.append(x)`	**O(1)***	Khấu hao
`a.pop()`	**O(1)**	Lấy phần tử cuối
`a.pop(0)`	**O(n)**	Dồn cả mảng
`a.insert(i, x)`	**O(n)**	Dời phần sau
`x in a`	**O(n)**	Quét lần lượt
`a[i:j]`	**O(j - i)**	Tạo list mới
`a.sort()`	**O(n log n)**	Timsort
`min`, `max`, `sum`	**O(n)**	Duyệt hết mảng

Lấy từ đầu liên tục: dùng `deque`. Kiểm tra `in` nhiều lần: dùng `set`.

9 Two Pointers: hai đầu tiến lại

Trên mảng đã sắp xếp, mỗi lần so sánh loại được một đầu. Từ $O(n^2)$ xuống $O(n)$.




```python
def two_sum_sorted(a, target):
    l, r = 0, len(a) - 1
    while l < r:
        s = a[l] + a[r]
        if s == target:
            return l, r
        if s < target:
            l += 1          # cần tổng lớn hơn
        else:
            r -= 1          # cần tổng nhỏ hơn
    return None

print(two_sum_sorted([1, 3, 4, 6, 8, 11], 10))
```

KẾT QUẢ KHI CHẠY

```
(2, 3)
```

- Vì sao đúng: `a[l] + a[r]` quá lớn thì `a[r]` không thể ghép với bất kỳ số nào từ `l` trở đi.
- Cùng dạng: kiểm tra palindrome, 3Sum, Container With Most Water.

10 Two Pointers: đọc và ghi

Một con trỏ đọc mọi phần tử, một con trỏ ghi phần được giữ lại. Sửa ngay trên mảng, bộ nhớ thêm O(1).

```
def remove_dup(a):                  # a đã sắp xếp
    w = 0                           # vị trí ghi tiếp theo
    for r in range(len(a)):         # r đọc từng phần tử
        if w == 0 or a[r] != a[w - 1]:
            a[w] = a[r]
            w += 1
    return a[:w]

def move_zeros(a):
    w = 0
    for r in range(len(a)):
        if a[r] != 0:
            a[w], a[r] = a[r], a[w]
            w += 1
    return a

print(remove_dup([1, 1, 2, 2, 2, 3]))
print(move_zeros([0, 1, 0, 3, 12]))
```

KẾT QUẢ KHI CHẠY

```
[1, 2, 3]
[1, 3, 12, 0, 0]
```

- ✓ Mỗi con trỏ chỉ đi tiến, nên tổng cộng O(n).
- ✓ Thứ tự các phần tử giữ lại không đổi.

11 Sliding Window

Giữ một đoạn liên tiếp: thêm bên phải, bớt bên trái, không tính lại từ đầu.

```python
def max_sum_k(a, k):                    # cửa sổ cố định
    s = best = sum(a[:k])
    for i in range(k, len(a)):
        s += a[i] - a[i - k]            # thêm 1, bớt 1
        best = max(best, s)
    return best

def longest_unique(s):                  # cửa sổ co giãn
    seen, l, best = {}, 0, 0
    for r, ch in enumerate(s):
        if ch in seen and seen[ch] >= l:
            l = seen[ch] + 1            # nhảy qua ký tự trùng
        seen[ch] = r
        best = max(best, r - l + 1)
    return best

print(max_sum_k([2, 1, 5, 1, 3, 2], 3))
print(longest_unique("abcabcbb"))
```

KẾT QUẢ KHI CHẠY

```
9
3
```

DẤU HIỆU

Đoạn con hoặc chuỗi con liên tiếp, dài nhất, ngắn nhất, thỏa điều kiện.

CẨN THẬN

Bài tổng có số âm thì cửa sổ co giãn không còn đúng, hãy dùng Prefix Sum.

12 Prefix Sum

Tính trước tổng tích lũy, sau đó mỗi truy vấn tổng đoạn chỉ tốn O(1).

```python
from itertools import accumulate

a = [3, 1, 4, 1, 5, 9]
p = [0] + list(accumulate(a))   # p[i]: tổng i phần tử đầu
print(p)

def range_sum(l, r):            # tổng a[l..r]
    return p[r + 1] - p[l]

print(range_sum(1, 3))
```

```python
def count_subarrays(a, k):      # số đoạn con có tổng k
    seen, s, count = {0: 1}, 0, 0
    for v in a:
        s += v
        count += seen.get(s - k, 0)
        seen[s] = seen.get(s, 0) + 1
    return count

print(count_subarrays([1, 1, 1], 2))
```

KẾT QUẢ KHI CHẠY

```
[0, 3, 4, 8, 9, 14, 23]
6
2
```

- Đoạn `l..r` có tổng k khi hai tổng tích lũy chênh nhau đúng k.
- Kết hợp với hash table cho bài có số âm, chạy trong O(n).

13 Chuỗi (String)

Chuỗi trong Python không sửa được tại chỗ. Mỗi lần nối là tạo chuỗi mới.

```python
from collections import Counter

words = ["dsa", "rất", "hay"]
print(" ".join(words))          # O(tổng độ dài)

def is_palindrome(s):
    s = [c.lower() for c in s if c.isalnum()]
    l, r = 0, len(s) - 1
    while l < r:
        if s[l] != s[r]:
            return False
        l, r = l + 1, r - 1
    return True

print(is_palindrome("A man, a plan, a canal: Panama"))
print(Counter("listen") == Counter("silent"))
```

KẾT QUẢ KHI CHẠY

```
dsa rất hay
True
True
```

- Ghép nhiều mảnh: gom vào list rồi `"".join`, tránh `s += c` trong vòng lặp lớn.
- Đếm ký tự: dùng `Counter` hoặc mảng 26 ô nếu chỉ có chữ thường.
- Hai kỹ thuật hay gặp nhất với chuỗi: Two Pointers và Sliding Window.

14 Ma trận

Mảng hai chiều là nền của bài lưới, bản đồ và bảng DP.

```
bad = [[0] * 3] * 2          # 2 hàng dùng chung một list
bad[0][0] = 1
print(bad)

good = [[0] * 3 for _ in range(2)]
good[0][0] = 1
print(good)

m = [[1, 2, 3], [4, 5, 6]]
print([list(col) for col in zip(*m)])       # chuyển vị

dirs = [(0, 1), (1, 0), (0, -1), (-1, 0)]  # 4 hướng
r, c = 0, 1
print([(r + dr, c + dc) for dr, dc in dirs
       if 0 <= r + dr < 2 and 0 <= c + dc < 3])
```

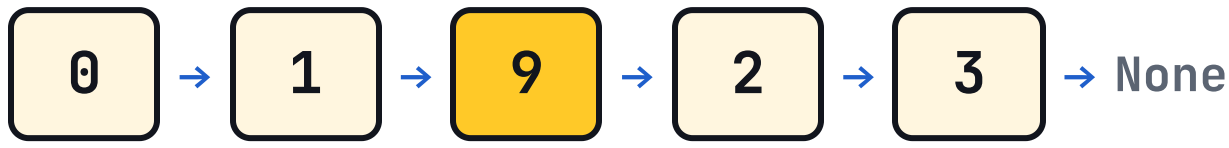
KẾT QUẢ KHI CHẠY

```
[[1, 0, 0], [1, 0, 0]]
[[1, 0, 0], [0, 0, 0]]
[[1, 4], [2, 5], [3, 6]]
[(0, 2), (1, 1), (0, 0)]
```

- ✓ Luôn tạo ma trận bằng list comprehension để mỗi hàng là một list riêng.
- ✓ Duyệt láng giềng bằng mảng hướng, kiểm tra biên trước khi truy cập.
- ✓ Ma trận R hàng, C cột: duyệt hết tốn O(R · C).

15 Danh sách liên kết

Mỗi nút giữ giá trị và con trỏ tới nút sau. Các nút nằm rải rác trong bộ nhớ.




```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

head = Node(1, Node(2, Node(3)))
head = Node(0, head)                # chèn đầu: O(1)

cur = head                          # tìm nút có giá trị 1
while cur.val != 1:
    cur = cur.next
cur.next = Node(9, cur.next)        # chèn sau nút đó: O(1)

print(to_list(head))
```

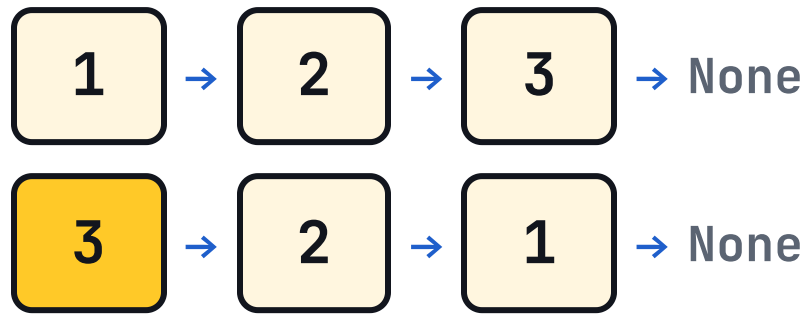
KẾT QUẢ KHI CHẠY

```
[0, 1, 9, 2, 3]
```

Thao tác	Mảng	Linked list
Truy cập phần tử thứ i	O(1)	O(n)
Chèn ở đầu	O(n)	O(1)
Chèn sau nút đã có	O(n)	O(1)

16 Đảo danh sách liên kết

Câu phỏng vấn kinh điển. Giữ 3 con trỏ và đảo từng mũi tên.




```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def reverse(head):
    prev = None
    while head:
        nxt = head.next          # giữ phần còn lại
        head.next = prev         # đảo mũi tên
        prev, head = head, nxt   # tiến một bước
    return prev

def show(h):
    out = []
    while h:
        out.append(h.val)
        h = h.next
    return out

print(show(reverse(Node(1, Node(2, Node(3))))))
```

KẾT QUẢ KHI CHẠY

```
[3, 2, 1]
```

- ✓ Thời gian O(n), bộ nhớ thêm O(1).
- ✓ Quên dòng `nxt` là mất toàn bộ phần sau của danh sách.

17 Nút giả: trộn hai danh sách

Một nút giả đứng trước đầu danh sách giúp khỏi phải xử lý riêng trường hợp nút đầu tiên.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def merge(a, b):
    dummy = tail = Node(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b           # nối phần còn lại
    return dummy.next

def build(xs):
    head = None
    for x in reversed(xs):
        head = Node(x, head)
    return head

r = merge(build([1, 3, 5]), build([2, 4]))
while r:
    print(r.val, end=" ")
    r = r.next
```

KẾT QUẢ KHI CHẠY

```
1 2 3 4 5
```

Khi code linked list mà cứ phải viết `if head is None` ở nhiều chỗ, hãy nghĩ tới nút giả.

18 Con trỏ nhanh và chậm

Con chậm đi 1 bước, con nhanh đi 2 bước. Có vòng thì hai con sẽ gặp nhau.

```
class Node:
    def __init__(self, val, next=None):
        self.val, self.next = val, next

def middle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    return slow.val

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False

a = Node(1, Node(2, Node(3, Node(4, Node(5)))))
print(middle(a), has_cycle(a))
a.next.next.next.next.next = a.next.next   # tạo vòng
print(has_cycle(a))
```

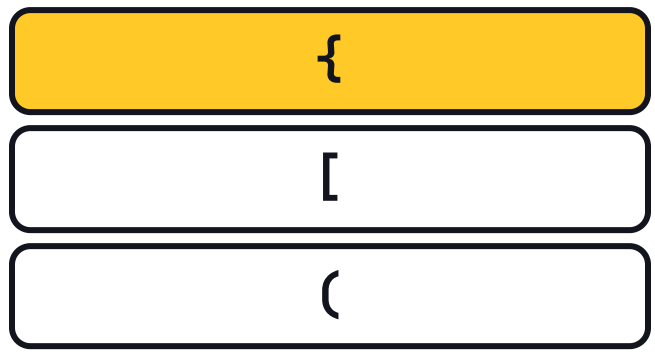
KẾT QUẢ KHI CHẠY

```
3 False
True
```

- ✓ Đây là thuật toán Floyd, thường gọi là thỏ và rùa. Bộ nhớ thêm O(1).
- ✓ Cùng dạng: tìm nút thứ k từ cuối, tìm điểm bắt đầu vòng.

19 Stack

Vào sau, ra trước. Dùng khi cần quay lại điều gần nhất.


{
[
(

```python
def valid(s):
    pair = {")": "(", "]": "[", "}": "{"}
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]:
                return False
        else:
            st.append(c)
    return not st

print(valid("{[()]}"), valid("("), valid("(("))
```

KẾT QUẢ KHI CHẠY

```
True False False
```

Ứng dụng	Stack lưu gì
Undo	Các thao tác vừa làm
Nút Back của trình duyệt	Các trang vừa xem
Gọi hàm	Khung của hàm đang chạy
Tính biểu thức	Toán hạng và toán tử

Trong Python, list làm stack rất tốt: `append` và `pop` đều O(1) ở cuối mảng.

20 Monotonic Stack

Giữ stack luôn tăng hoặc giảm để tìm phần tử lớn hơn gần nhất trong O(n).

```python
def next_greater(a):
    res = [-1] * len(a)
    st = []                          # chỉ số, giá trị giảm dần
    for i, v in enumerate(a):
        while st and a[st[-1]] < v:
            res[st.pop()] = v        # v lớn hơn đầu tiên
        st.append(i)
    return res

def largest_rect(h):                 # hình chữ nhật lớn nhất
    st, best = [], 0
    for i, x in enumerate(h + [0]):
        while st and h[st[-1]] >= x:
            top = h[st.pop()]
            left = st[-1] if st else -1
            best = max(best, top * (i - left - 1))
        st.append(i)
    return best

print(next_greater([2, 1, 2, 4, 3]))
print(largest_rect([2, 1, 5, 6, 2, 3]))
```

KẾT QUẢ KHI CHẠY

```
[4, 2, 4, -1, -1]
10
```

- Mỗi chỉ số vào stack một lần và ra một lần, nên tổng cộng O(n) dù có vòng `while` bên trong.
- Cùng dạng: Daily Temperatures, Trapping Rain Water, Stock Span.

21 Queue và Deque

Vào trước, ra trước. Dùng cho xếp hàng và duyệt theo tầng.

```
from collections import deque

q = deque()
q.append("An")          # vào cuối: O(1)
q.append("Bình")
print(q.popleft())      # ra đầu: O(1)

class CircularQueue:    # hàng đợi vòng trên mảng cố định
    def __init__(self, k):
        self.a, self.head, self.n = [None] * k, 0, 0

    def push(self, x):
        if self.n == len(self.a):
            return False
        self.a[(self.head + self.n) % len(self.a)] = x
        self.n += 1
        return True

    def pop(self):
        x = self.a[self.head]
        self.head = (self.head + 1) % len(self.a)
        self.n -= 1
        return x

c = CircularQueue(2)
print(c.push(1), c.push(2), c.push(3), c.pop(), c.push(3))
```

KẾT QUẢ KHI CHẠY

```
An
True True False 1 True
```

- Đừng dùng `list.pop(0)` làm hàng đợi: mỗi lần tốn O(n).
- `deque` thêm và bớt ở cả hai đầu đều O(1).

22 Lớn nhất trong cửa sổ

Deque chỉ giữ những ứng viên còn có cơ hội làm giá trị lớn nhất.

```
from collections import deque

def window_max(a, k):
    q, res = deque(), []          # q lưu chỉ số, giá trị giảm dần
    for i, v in enumerate(a):
        while q and a[q[-1]] <= v:
            q.pop()               # số nhỏ hơn hết cơ hội
        q.append(i)
        if q[0] <= i - k:
            q.popleft()           # đã ra khỏi cửa sổ
        if i >= k - 1:
            res.append(a[q[0]])
    return res

print(window_max([1, 3, -1, -3, 5, 3, 6, 7], 3))
```

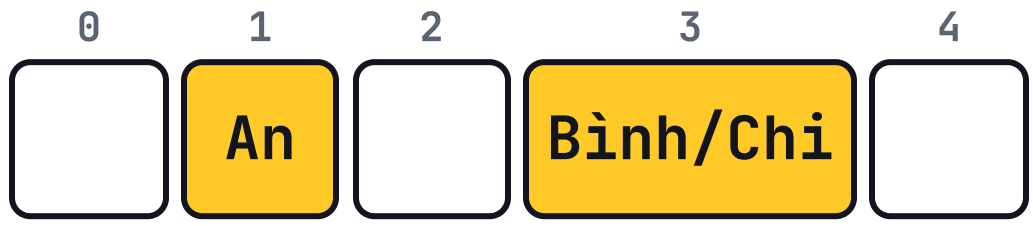
KẾT QUẢ KHI CHẠY

```
[3, 3, 5, 5, 6, 7]
```

- Cách thường: mỗi cửa sổ tìm max lại, tốn O(n · k).
- Cách này: mỗi phần tử vào và ra deque một lần, tốn O(n).
- Đầu deque luôn là giá trị lớn nhất của cửa sổ hiện tại.

23 Hash table hoạt động ra sao?

Hàm băm biến khóa thành một số. Số đó chỉ ra ô cần tới trong mảng, nên không phải quét từng phần tử.




```python
from timeit import timeit

ids = list(range(1_000_000))
s = set(ids)
t_list = timeit(lambda: 999_999 in ids, number=20)
t_set = timeit(lambda: 999_999 in s, number=20)
print(t_list > 1000 * t_set)    # set nhanh hơn trên 1.000 lần
```

KẾT QUẢ KHI CHẠY

```
True
```

Khái niệm	Ý nghĩa
Va chạm	Hai khóa rơi vào cùng một ô, như Bình và Chi ở ô 3
Xâu chuỗi	Mỗi ô giữ một danh sách các phần tử
Dò địa chỉ mở	Ô đã có người thì tìm ô trống khác
Hệ số tải	Số phần tử chia số ô. Quá cao thì mở rộng bảng

- ✓ Tìm, thêm, xóa trung bình O(1). Xấu nhất O(n) nếu quá nhiều va chạm.
- ✓ Khóa phải băm được: `tuple` làm khóa được, `list` thì không.
- ✓ Đây là lời giải cho câu đố ở trang bìa.

24 Tự viết một HashMap

Cài bằng xâu chuỗi, quá đầy thì gấp đôi số ô.

```
class HashMap:
    def __init__(self, size=8):
        self.b = [[] for _ in range(size)]
        self.n = 0
    def _slot(self, key):
        return self.b[hash(key) % len(self.b)]

    def put(self, key, val):
        for pair in self._slot(key):
            if pair[0] == key:              # khóa đã có: cập nhật
                pair[1] = val
                return
        self._slot(key).append([key, val])
        self.n += 1
        if self.n > len(self.b) * 0.75:     # quá đầy: mở rộng
            old = self.b
            self.b = [[] for _ in range(2 * len(old))]
            for slot in old:
                for k, v in slot:
                    self._slot(k).append([k, v])

    def get(self, key, default=None):
        for k, v in self._slot(key):
            if k == key:
                return v
        return default

m = HashMap()
for i in range(20):
    m.put(f"k{i}", i)
m.put("k3", 99)
print(m.get("k3"), m.get("k19"), m.get("x"), len(m.b))
```

KẾT QUẢ KHI CHẠY

```
99 19 None 32
```

25 Dict, Set, Counter thực chiến

Đổi bộ nhớ lấy tốc độ: nhớ những gì đã thấy để khỏi phải tìm lại.

```python
from collections import Counter, defaultdict

def two_sum(a, target):                # O(n) thay vì O(n²)
    seen = {}
    for i, v in enumerate(a):
        if target - v in seen:
            return seen[target - v], i
        seen[v] = i

def group_anagrams(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())

print(two_sum([2, 7, 11, 15], 9))
print(group_anagrams(["eat", "tea", "tan", "ate", "nat"]))
print(Counter("mississippi").most_common(2))
```

KẾT QUẢ KHI CHẠY

```
(0, 1)
[['eat', 'tea', 'ate'], ['tan', 'nat']]
[('i', 4), ('s', 4)]
```

Cần làm gì	Dùng
Kiểm tra đã thấy chưa, loại trùng	**set**
Ánh xạ khóa sang giá trị	**dict**
Đếm tần suất	**Counter**
Gom nhóm theo khóa	**defaultdict(list)**

26 LRU Cache

Bộ nhớ đệm đầy thì bỏ phần tử lâu chưa dùng nhất. Câu hỏi thiết kế rất hay gặp.

```python
from collections import OrderedDict

class LRU:
    def __init__(self, cap):
        self.cap, self.d = cap, OrderedDict()

    def get(self, k):
        if k not in self.d:
            return -1
        self.d.move_to_end(k)              # vừa dùng xong
        return self.d[k]

    def put(self, k, v):
        self.d[k] = v
        self.d.move_to_end(k)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)     # bỏ phần tử cũ nhất

c = LRU(2)
c.put("a", 1)
c.put("b", 2)
c.get("a")
c.put("c", 3)                              # "b" bị loại
print(list(c.d), c.get("b"))
```

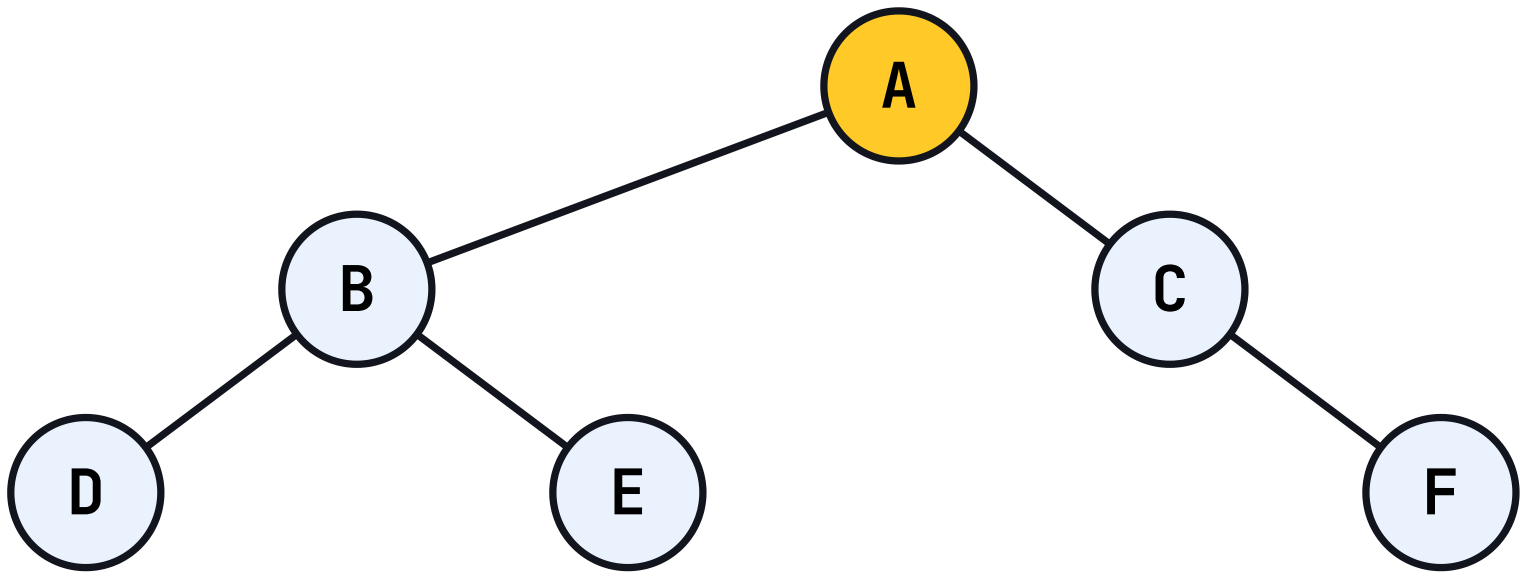
KẾT QUẢ KHI CHẠY

```
['a', 'c'] -1
```

- ✓ `get` và `put` đều O(1).
- ✓ Phỏng vấn thường yêu cầu tự cài: hash table để tìm nút, danh sách liên kết đôi để giữ thứ tự dùng.

27 Cây: thuật ngữ cơ bản

Cây gồm các nút nối theo quan hệ cha con, không có vòng. Mỗi nút trừ gốc có đúng một cha.

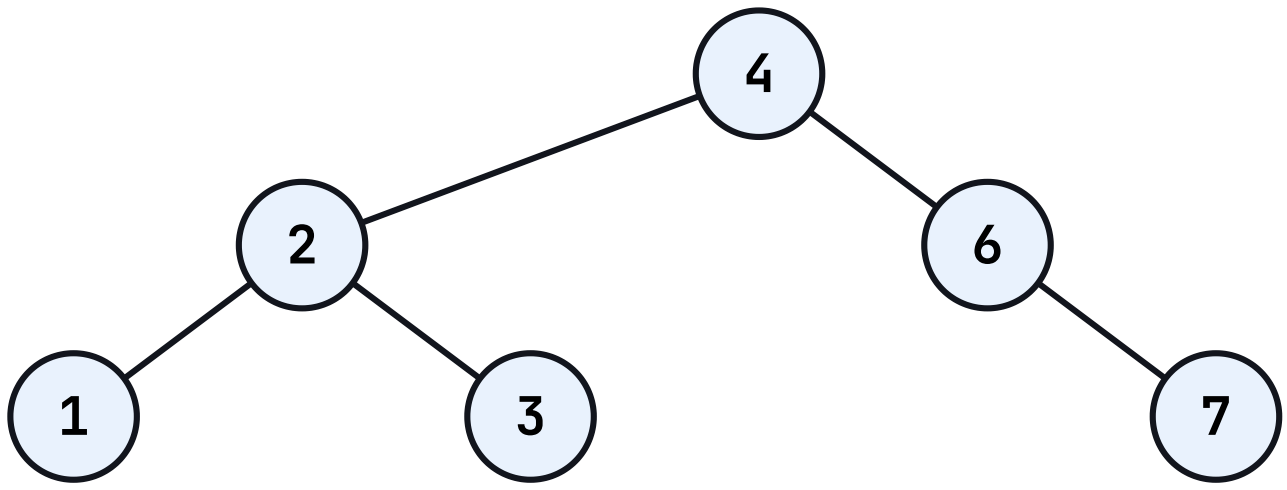


Thuật ngữ	Trong hình
Gốc	A, nút không có cha
Lá	D, E, F, nút không có con
Cha, con	B là cha của D và E
Độ sâu	Số cạnh từ gốc tới nút: D có độ sâu 2
Chiều cao	Đường dài nhất từ gốc xuống lá: 2 cạnh
Cây nhị phân	Mỗi nút có tối đa 2 con

Có tài liệu đếm chiều cao theo số nút, nên đọc kỹ định nghĩa trong đề.

28 4 cách duyệt cây

Pre, in, post chỉ vị trí của nút gốc: đứng trước, giữa, hay sau hai con.




```
from collections import deque
class T:
    def __init__(self, v, l=None, r=None):
        self.v, self.l, self.r = v, l, r

root = T(4, T(2, T(1), T(3)), T(6, None, T(7)))
def pre(t):  return [t.v] + pre(t.l) + pre(t.r) if t else []
def ino(t):  return ino(t.l) + [t.v] + ino(t.r) if t else []
def post(t): return post(t.l) + post(t.r) + [t.v] if t else []

def level(t):                          # BFS theo từng tầng
    q, out = deque([t]), []
    while q:
        x = q.popleft()
        out.append(x.v)
        q.extend(c for c in (x.l, x.r) if c)
    return out

print(pre(root), ino(root))
print(post(root), level(root))
```

KẾT QUẢ KHI CHẠY

```
[4, 2, 1, 3, 6, 7] [1, 2, 3, 4, 6, 7]
[1, 3, 2, 7, 6, 4] [4, 2, 6, 1, 3, 7]
```

Mỗi nút được thăm một lần. Bản nối list này ngắn nhưng tốn hơn O(n).

29 Tư duy đệ quy trên cây

Giả sử hàm đã đúng với hai cây con, chỉ cần nghĩ cách ghép kết quả tại nút hiện tại.

```
class T:
    def __init__(self, v, l=None, r=None):
        self.v, self.l, self.r = v, l, r

def height(t):                  # đếm theo số nút
    return 0 if not t else 1 + max(height(t.l), height(t.r))

def same(a, b):
    if not a or not b:
        return a is b
    return a.v == b.v and same(a.l, b.l) and same(a.r, b.r)

def invert(t):
    if t:
        t.l, t.r = invert(t.r), invert(t.l)
    return t

def lca(t, p, q):               # tổ tiên chung gần nhất
    if not t or t.v in (p, q):
        return t
    l, r = lca(t.l, p, q), lca(t.r, p, q)
    return t if l and r else l or r

root = T(4, T(2, T(1), T(3)), T(6, None, T(7)))
print(height(root), same(root, root), lca(root, 1, 3).v)
print(invert(root).l.v)
```

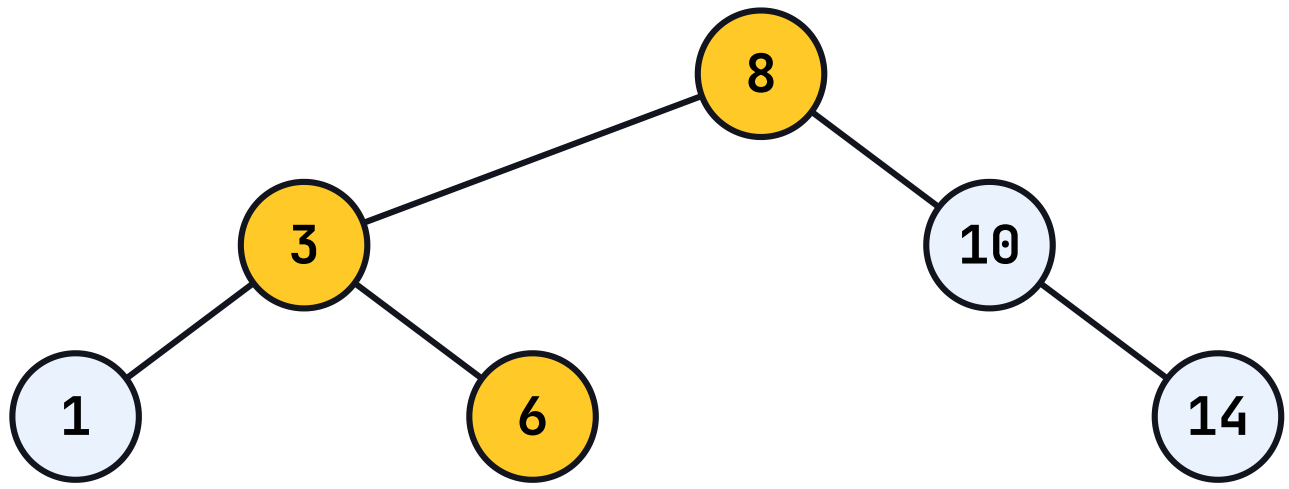
KẾT QUẢ KHI CHẠY

```
3 True 2
6
```

Mẫu chung: xử lý nút rỗng, gọi đệ quy hai con, ghép kết quả. Hầu hết bài cây đều theo khung này.

30 Cây nhị phân tìm kiếm (BST)

Mọi nút bên trái nhỏ hơn, bên phải lớn hơn. Mỗi bước loại được một nhánh, duyệt in-order sẽ ra dãy tăng dần.




```
class T:
    def __init__(self, v):
        self.v, self.l, self.r = v, None, None
def insert(t, v):
    if not t:
        return T(v)
    if v < t.v:
        t.l = insert(t.l, v)
    elif v > t.v:
        t.r = insert(t.r, v)
    return t                        # trùng thì bỏ qua
def search(t, v):
    while t and t.v != v:
        t = t.l if v < t.v else t.r
    return t is not None
root = None
for x in [8, 3, 10, 1, 6, 14]:
    root = insert(root, x)
print(search(root, 6), search(root, 7))
```

KẾT QUẢ KHI CHẠY

```
True False
```

31 Xóa nút trong BST

Khó nhất là nút có 2 con: thay bằng nút nhỏ nhất của nhánh phải, rồi xóa nút đó.

```
class T:
    def __init__(self, v, l=None, r=None):
        self.v, self.l, self.r = v, l, r
def delete(t, v):
    if not t:
        return None
    if v < t.v:
        t.l = delete(t.l, v)
    elif v > t.v:
        t.r = delete(t.r, v)
    elif not t.l:                  # 0 hoặc 1 con
        return t.r
    elif not t.r:
        return t.l
    else:                          # 2 con
        m = t.r
        while m.l:
            m = m.l
        t.v = m.v
        t.r = delete(t.r, m.v)
    return t
def inorder(t):
    return inorder(t.l) + [t.v] + inorder(t.r) if t else []

root = T(8, T(3, T(1), T(6)), T(10, None, T(14)))
print(inorder(delete(root, 3)), root.l.v)
```

KẾT QUẢ KHI CHẠY

```
[1, 6, 8, 10, 14] 6
```

- ✓ Nút lá: xóa luôn. Nút một con: đưa con lên thay.
- ✓ Nút thay thế lớn hơn mọi nút bên trái, nên cây vẫn đúng thứ tự.
- ✓ Chi phí O(chiều cao): cây cân bằng là O(log n), cây lệch là O(n).

32 Cây cân bằng

BST lệch tụt từ O(log n) xuống O(n). Cây cân bằng tự xoay để giữ chiều cao thấp.

```
class T:
    def __init__(self, v):
        self.v, self.l, self.r = v, None, None
def insert(t, v):
    if not t:
        return T(v)
    if v < t.v:
        t.l = insert(t.l, v)
    else:
        t.r = insert(t.r, v)
    return t
def height(t):
    return 0 if not t else 1 + max(height(t.l), height(t.r))

def build(order):
    t = None
    for x in order:
        t = insert(t, x)
    return height(t)

print(build(range(1, 8)))                # thành đường thẳng
print(build([4, 2, 6, 1, 3, 5, 7]))      # cân đối
```

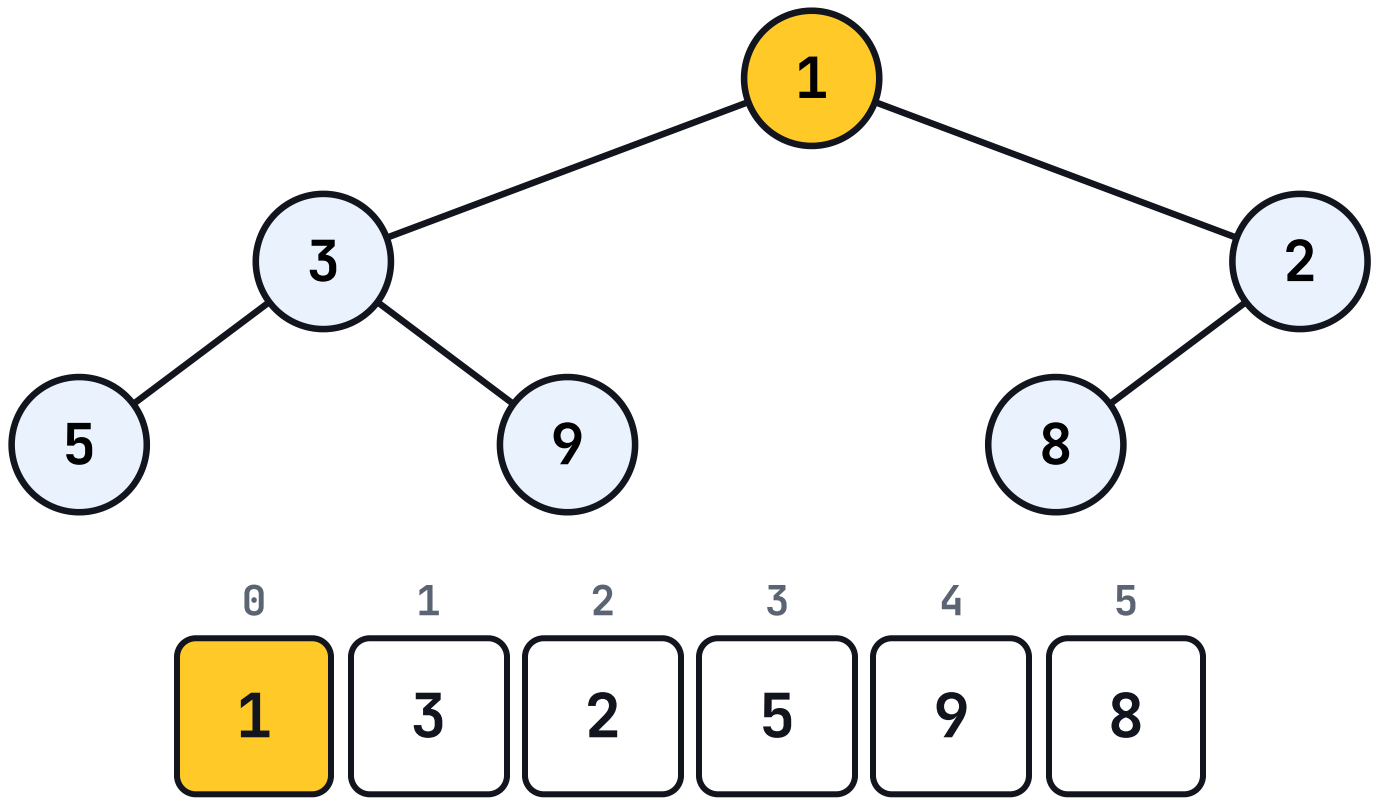
KẾT QUẢ KHI CHẠY

```
7
3
```

Loại cây	Ý tưởng	Thường gặp ở
AVL	Hai nhánh chênh tối đa 1	Tra cứu nhiều
Đỏ đen	Tô màu, nới lỏng cân bằng	Java TreeMap, C++ map
B+Tree	Một nút chứa nhiều khóa	Index của database

33 Heap

Cây nhị phân lưu trong mảng. Với min-heap, cha luôn nhỏ hơn hoặc bằng con, nên phần tử nhỏ nhất ở đầu.



TRONG MẢNG, NÚT i CÓ
Cha ở `(i - 1) // 2`, con trái ở `2 * i + 1`, con phải ở `2 * i + 2`.

Thao tác	Cách làm	Chi phí
Xem đỉnh	Đọc `a[0]`	O(1)
Thêm	Đặt cuối mảng rồi cho nổi lên	O(log n)
Lấy đỉnh	Đưa phần tử cuối lên đầu rồi cho chìm xuống	O(log n)
Dựng từ mảng	`heapify`	O(n)

Heap không sắp xếp toàn bộ. Nó chỉ đảm bảo đỉnh đúng, nên nhanh hơn việc giữ cả dãy có thứ tự.

34 Tự viết Min-Heap

Thêm thì nổi lên, lấy ra thì chìm xuống. Chiều cao là log n nên mỗi lần tốn O(log n).

```
class MinHeap:
    def __init__(self):
        self.a = []
    def push(self, x):
        a = self.a
        a.append(x)
        i = len(a) - 1
        while i and a[(i - 1) // 2] > a[i]:      # nổi lên
            a[(i - 1) // 2], a[i] = a[i], a[(i - 1) // 2]
            i = (i - 1) // 2

    def pop(self):
        a = self.a
        a[0], a[-1] = a[-1], a[0]
        top = a.pop()
        i = 0
        while True:                              # chìm xuống
            c = 2 * i + 1
            if c + 1 < len(a) and a[c + 1] < a[c]:
                c += 1
            if c >= len(a) or a[i] <= a[c]:
                return top
            a[i], a[c] = a[c], a[i]
            i = c

h = MinHeap()
for x in [5, 3, 8, 1, 9, 2]:
    h.push(x)
print(h.a)
print([h.pop() for _ in range(6)])
```

KẾT QUẢ KHI CHẠY

```
[1, 3, 2, 5, 9, 8]
[1, 2, 3, 5, 8, 9]
```

35 heapq thực chiến

Dùng heap khi liên tục cần phần tử nhỏ nhất hoặc lớn nhất, nhưng không cần sắp xếp toàn bộ.

```
import heapq

a = [7, 2, 9, 4, 1, 8]
print(heapq.nlargest(2, a), heapq.nsmallest(2, a))

def top_k(a, k):                          # O(n log k)
    h = []
    for x in a:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)              # bỏ số nhỏ nhất
    return sorted(h, reverse=True)

print(top_k(a, 3))
print(list(heapq.merge([1, 4], [2, 5], [3, 6])))
```

KẾT QUẢ KHI CHẠY

```
[9, 8] [1, 2]
[9, 8, 7]
[1, 2, 3, 4, 5, 6]
```

- ✓ `heapq` chỉ có min-heap. Muốn max-heap thì lưu số âm.
- ✓ Giữ heap cỡ k tốt hơn sắp xếp cả mảng khi k nhỏ.
- ✓ Trộn k danh sách đã sắp xếp: O(n log k).
- ✓ Heap chứa tuple thì so sánh lần lượt từng phần tử, rất tiện cho bài có độ ưu tiên.

36 Trung vị của dòng dữ liệu

Hai heap chia dữ liệu thành nửa nhỏ và nửa lớn, trung vị nằm ngay ở đỉnh.

```python
import heapq

class Median:
    def __init__(self):
        self.lo, self.hi = [], []    # lo là max-heap nhờ đổi dấu

    def add(self, x):                # O(log n)
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def get(self):                   # O(1)
        if len(self.lo) > len(self.hi):
            return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2

m = Median()
for x in [5, 2, 8, 4]:
    m.add(x)
print(m.get())
```

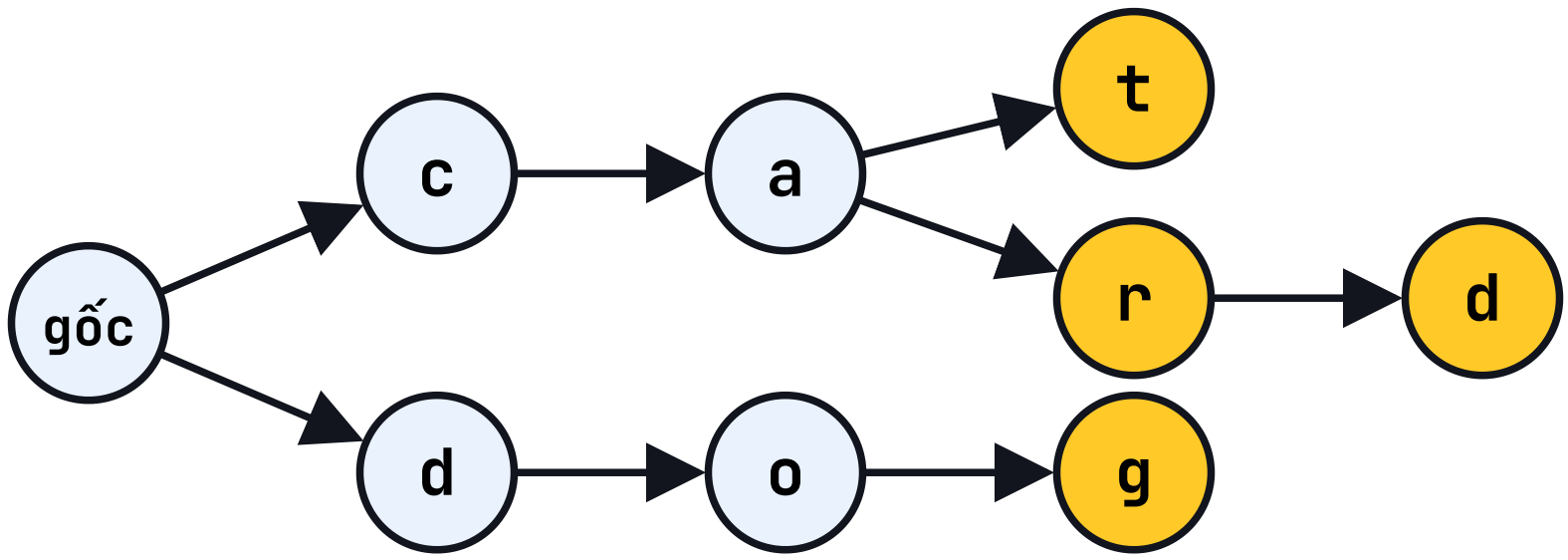
KẾT QUẢ KHI CHẠY

```
4.5
```

- ✓ `lo` giữ nửa nhỏ, `hi` giữ nửa lớn, số phần tử chênh tối đa 1.
- ✓ Sắp xếp lại sau mỗi số mới sẽ tốn nhiều hơn rất nhiều.

37 Trie

Cây lưu từng ký tự. Các từ có chung tiền tố dùng chung đường đi, nên rất hợp cho gợi ý tìm kiếm.



Lưu 4 từ: cat, car, card, dog. Nút vàng là nơi kết thúc một từ.

Tiêu chí	Trie	Hash table
Tìm đúng một từ	O(L)	O(L) để băm chuỗi
Tìm theo tiền tố	Đi thẳng tới nhánh	Phải quét cả bảng
Thứ tự từ điển	Có sẵn khi duyệt	Không có
Bộ nhớ	Tốn hơn	Gọn hơn

L là độ dài của từ, không phụ thuộc số từ đã lưu.

38 Cài đặt Trie và gợi ý từ

Dùng dict lồng nhau, ký tự `$` đánh dấu nơi kết thúc một từ.

```
class Trie:
    def __init__(self):
        self.root = {}
    def add(self, word):
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node["$"] = True
    def find(self, s):                  # đi tới nút của tiền tố s
        node = self.root
        for c in s:
            node = node.get(c)
            if node is None:
                return None
        return node
    def suggest(self, prefix):
        out = []
        def walk(n, s):
            for c, nxt in sorted(n.items()):
                if c == "$":
                    out.append(s)
                else:
                    walk(nxt, s + c)
        walk(self.find(prefix) or {}, prefix)
        return out

t = Trie()
for w in ["cat", "car", "card", "dog"]:
    t.add(w)
print("$" in t.find("car"), "$" in t.find("ca"))
print(t.suggest("car"), t.suggest("x"))
```

KẾT QUẢ KHI CHẠY

```
True False
['car', 'card'] []
```

39 Union-Find

Gộp hai nhóm và hỏi hai phần tử có cùng nhóm không, gần như O(1).

```
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.size = [1] * n

    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]   # nén đường đi
            x = self.p[x]
        return x

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.p[b] = a                       # cây nhỏ gắn vào cây lớn
        self.size[a] += self.size[b]
        return True

d = DSU(5)
d.union(0, 1)
d.union(3, 4)
print(d.find(0) == d.find(1), d.find(1) == d.find(3))
print(d.union(1, 0))                        # đã cùng nhóm
print(len({d.find(i) for i in range(5)}))
```

KẾT QUẢ KHI CHẠY

```
True False
False
3
```

Dùng cho: đếm thành phần liên thông, phát hiện vòng, thuật toán Kruskal.

40 Fenwick Tree

Prefix Sum không cập nhật được nhanh. Fenwick Tree làm được cả sửa số lẫn tính tổng trong O(log n).

```
class BIT:
    def __init__(self, n):
        self.t = [0] * (n + 1)

    def add(self, i, v):          # i tính từ 1
        while i < len(self.t):
            self.t[i] += v
            i += i & -i           # lên nút quản lý tiếp theo

    def sum(self, i):             # tổng từ 1 đến i
        s = 0
        while i:
            s += self.t[i]
            i -= i & -i           # bỏ bit thấp nhất
        return s

a = [3, 2, 5, 1, 4]
b = BIT(len(a))
for i, v in enumerate(a, 1):
    b.add(i, v)
print(b.sum(3), b.sum(5) - b.sum(2))
b.add(2, 10)                      # phần tử thứ 2 tăng 10
print(b.sum(3))
```

KẾT QUẢ KHI CHẠY

```
10 10
20
```

- `i & -i` lấy bit 1 thấp nhất của i, quyết định mỗi ô quản lý đoạn dài bao nhiêu.
- Mảng tính từ chỉ số 1. Đây là lỗi hay gặp nhất khi cài đặt.

41 Segment Tree

Mỗi nút quản lý một đoạn. Dùng được cho min, max và tổng.

```
class SegTree:                          # min trên một đoạn
    def __init__(self, a):
        self.n = len(a)
        self.t = [0] * self.n + a
        for i in range(self.n - 1, 0, -1):
            self.t[i] = min(self.t[2 * i], self.t[2 * i + 1])
    def update(self, i, v):
        i += self.n
        self.t[i] = v
        while i > 1:
            i //= 2
            self.t[i] = min(self.t[2 * i], self.t[2 * i + 1])

    def query(self, l, r):              # min của a[l..r-1]
        res = float("inf")
        l, r = l + self.n, r + self.n
        while l < r:
            if l % 2:
                res = min(res, self.t[l])
                l += 1
            if r % 2:
                r -= 1
                res = min(res, self.t[r])
            l, r = l // 2, r // 2
        return res

s = SegTree([5, 2, 6, 3, 7])
print(s.query(0, 5), s.query(2, 4))
s.update(3, 1)
print(s.query(2, 4))
```

KẾT QUẢ KHI CHẠY

```
2 3
1
```

Truy vấn và cập nhật đều O(log n). Mảng lưu 2n ô, nút i có con là 2i và 2i + 1.

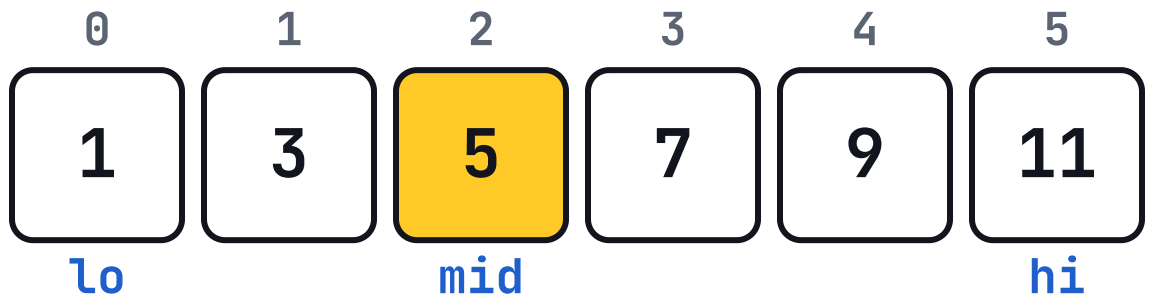
Những cấu trúc nên biết tên

Không cần tự cài ngay, nhưng nên biết chúng giải quyết việc gì.

Tên	Giải quyết việc gì
Bloom Filter	Kiểm tra "chắc chắn chưa có" với rất ít bộ nhớ, có thể báo nhầm là đã có
Skip List	Danh sách nhiều tầng, tìm kiếm O(log n) trung bình. Redis dùng cho sorted set
Sparse Table	Truy vấn min, max O(1) trên mảng không đổi
Suffix Array	Xử lý chuỗi con trên văn bản lớn
Consistent Hashing	Chia dữ liệu cho nhiều máy, thêm máy ít phải dời dữ liệu
LSM Tree	Tối ưu ghi cho một số database, như RocksDB, Cassandra

42 Tìm kiếm nhị phân

Trên dữ liệu đã sắp xếp, mỗi lần so sánh bỏ được một nửa. Một tỷ phần tử chỉ cần khoảng 30 bước.




```python
def binary_search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if a[mid] < x:
            lo = mid + 1       # bỏ nửa trái
        else:
            hi = mid - 1       # bỏ nửa phải
    return -1

a = [1, 3, 5, 7, 9, 11]
print(binary_search(a, 7), binary_search(a, 4))
```

KẾT QUẢ KHI CHẠY

```
3 -1
```

- ✓ Điều kiện bắt buộc: dữ liệu đã có thứ tự.
- ✓ Luôn cộng trừ 1 khi thu hẹp, nếu không rất dễ lặp vô hạn.
- ✓ Trong Java, C++: viết `lo + (hi - lo) / 2` để tránh tràn số.

43 Cận dưới và cận trên

Với dữ liệu có phần tử trùng, cần tìm vị trí đầu tiên thỏa điều kiện chứ không chỉ một vị trí bất kỳ.

```
from bisect import bisect_left, bisect_right, insort

def lower_bound(a, x):       # vị trí đầu tiên có a[i] >= x
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
    return lo

a = [1, 2, 2, 2, 5, 8]
print(lower_bound(a, 2), bisect_left(a, 2), bisect_right(a, 2))
print(bisect_right(a, 2) - bisect_left(a, 2))   # đếm số 2
insort(a, 3)                    # chèn mà vẫn giữ thứ tự
print(a)
```

KẾT QUẢ KHI CHẠY

```
1 1 4
3
[1, 2, 2, 2, 3, 5, 8]
```

Hàm	Trả về
`bisect_left(a, x)`	Vị trí đầu tiên có `a[i] >= x`
`bisect_right(a, x)`	Vị trí đầu tiên có `a[i] > x`
`insort(a, x)`	Tìm O(log n), nhưng chèn vẫn O(n)

44 Tìm trong mảng bị xoay

Mảng sắp xếp bị xoay vẫn tìm được trong O(log n), vì luôn có một nửa còn nguyên thứ tự.




```python
def search_rotated(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        if a[lo] <= a[mid]:          # nửa trái có thứ tự
            if a[lo] <= x < a[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:                        # nửa phải có thứ tự
            if a[mid] < x <= a[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1

print(search_rotated([4, 5, 6, 7, 0, 1, 2], 0))
```

KẾT QUẢ KHI CHẠY

```
4
```

Lời giải này giả định các phần tử khác nhau. Có phần tử trùng thì xấu nhất có thể về O(n).

45 Tìm nhị phân theo đáp án

Không tìm trong mảng mà tìm trên khoảng đáp án. Dùng khi đáp án càng lớn thì càng dễ đạt.

```
def can_ship(w, days, cap):        # sức chở cap có đủ không?
    need, load = 1, 0
    for x in w:
        if load + x > cap:
            need, load = need + 1, 0
        load += x
    return need <= days

def min_capacity(w, days):
    lo, hi = max(w), sum(w)
    while lo < hi:
        mid = (lo + hi) // 2
        if can_ship(w, days, mid):
            hi = mid               # chở được, thử nhỏ hơn
        else:
            lo = mid + 1
    return lo

print(min_capacity([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 5))
```

KẾT QUẢ KHI CHẠY

```
15
```

DẤU HIỆU
Đề hỏi giá trị nhỏ nhất sao cho, hoặc lớn nhất sao cho.

CHI PHÍ
O(n log S), với S là độ rộng khoảng đáp án.

Cùng dạng: Koko Eating Bananas, Split Array Largest Sum, Aggressive Cows.

46 Ba cách sắp xếp O(n²)

Chậm với dữ liệu lớn, nhưng là nền để hiểu các thuật toán sau. Insertion sort vẫn rất tốt với mảng nhỏ.

```python
def bubble(a):                  # đẩy số lớn về cuối
    for i in range(len(a) - 1, 0, -1):
        swapped = False
        for j in range(i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        if not swapped:         # không đổi chỗ thì đã xong
            break
    return a

def selection(a):               # chọn số nhỏ nhất đặt lên đầu
    for i in range(len(a)):
        m = min(range(i, len(a)), key=a.__getitem__)
        a[i], a[m] = a[m], a[i]
    return a

def insertion(a):               # chèn vào phần đã sắp xếp
    for i in range(1, len(a)):
        x, j = a[i], i - 1
        while j >= 0 and a[j] > x:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = x
    return a

d = [5, 2, 4, 1, 3]
print(bubble(d[:]), selection(d[:]), insertion(d[:]))
```

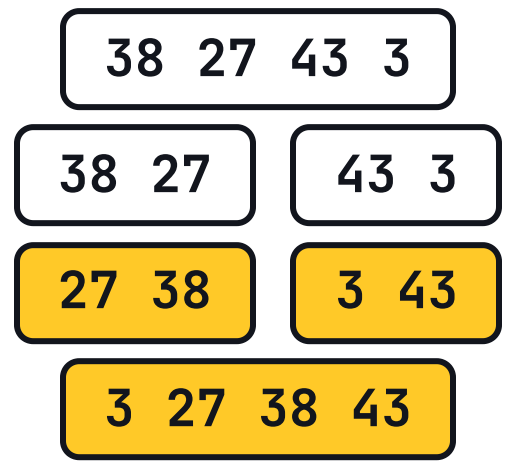
KẾT QUẢ KHI CHẠY

```
[1, 2, 3, 4, 5] [1, 2, 3, 4, 5] [1, 2, 3, 4, 5]
```

Mảng gần như đã sắp xếp: Insertion sort gần O(n). Vì vậy nhiều thuật toán thực tế kết hợp nó với merge sort hoặc quick sort.

47 Merge Sort

Chia đôi, sắp xếp từng nửa, rồi trộn lại. Luôn O(n log n).




```python
def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    l, r = merge_sort(a[:mid]), merge_sort(a[mid:])
    out, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] <= r[j]:          # dùng <= để giữ tính ổn định
            out.append(l[i])
            i += 1
        else:
            out.append(r[j])
            j += 1
    return out + l[i:] + r[j:]

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

KẾT QUẢ KHI CHẠY

```
[3, 9, 10, 27, 38, 43, 82]
```

- ✓ Có log n tầng, mỗi tầng trộn tổng cộng n phần tử.
- ✓ Ổn định, nhưng cần bộ nhớ thêm O(n).
- ✓ Hợp với linked list và sắp xếp dữ liệu quá lớn so với RAM.

48 Quick Sort

Chọn chốt, đưa số nhỏ sang trái, số lớn sang phải, rồi làm tiếp từng bên.

```python
import random

def quick_sort(a, lo=0, hi=None):
    if hi is None:
        hi = len(a) - 1
    if lo >= hi:
        return a
    p = random.randint(lo, hi)          # chốt ngẫu nhiên
    a[p], a[hi] = a[hi], a[p]
    pivot, i = a[hi], lo
    for j in range(lo, hi):             # phân hoạch Lomuto
        if a[j] < pivot:
            a[i], a[j] = a[j], a[i]
            i += 1
    a[i], a[hi] = a[hi], a[i]           # chốt về đúng chỗ
    quick_sort(a, lo, i - 1)
    quick_sort(a, i + 1, hi)
    return a

print(quick_sort([9, 4, 7, 1, 8, 2]))
```

KẾT QUẢ KHI CHẠY

```
[1, 2, 4, 7, 8, 9]
```

TRUNG BÌNH
O(n log n), sắp xếp ngay trên mảng.

XẤU NHẤT
O(n²) nếu chốt luôn lệch. Chốt ngẫu nhiên giúp tránh.

Mảng có rất nhiều phần tử trùng thì nên chia làm 3 phần: nhỏ hơn, bằng, lớn hơn chốt.

49 Quickselect

Tìm số nhỏ thứ k mà không cần sắp xếp hết: sau khi phân hoạch, chỉ đi tiếp vào một bên.

```python
import random

def kth_smallest(a, k):
    p = random.choice(a)
    lo = [x for x in a if x < p]
    eq = [x for x in a if x == p]
    hi = [x for x in a if x > p]
    if k <= len(lo):
        return kth_smallest(lo, k)
    if k <= len(lo) + len(eq):
        return p
    return kth_smallest(hi, k - len(lo) - len(eq))

print(kth_smallest([9, 4, 7, 1, 8, 2], 2))
```

KẾT QUẢ KHI CHẠY

2

Cách làm	Chi phí	Khi nào dùng
Sắp xếp	O(n log n)	Code ngắn, n vừa phải
Heap cỡ k	O(n log k)	Dữ liệu đến theo dòng
Quickselect	O(n) trung bình	Chỉ cần một phần tử thứ k

Nếu mỗi lần bỏ được khoảng một nửa, tổng chi phí là n + n/2 + n/4 + ... tức khoảng 2n.

50 Heap, Counting, Radix Sort

So sánh không thể nhanh hơn O(n log n). Counting và Radix vượt được nhờ dùng giá trị số nguyên.

```
import heapq
def heap_sort(a):
    h = a[:]
    heapq.heapify(h)                # O(n)
    return [heapq.heappop(h) for _ in a]

def counting_sort(a, k):            # giá trị trong 0..k
    c = [0] * (k + 1)
    for x in a:
        c[x] += 1
    return [v for v in range(k + 1) for _ in range(c[v])]

def radix_sort(a):                  # số nguyên không âm
    exp = 1
    while a and max(a) // exp:
        b = [[] for _ in range(10)]
        for x in a:
            b[x // exp % 10].append(x)
        a = [x for bucket in b for x in bucket]
        exp *= 10
    return a

print(heap_sort([4, 1, 3]), counting_sort([3, 0, 2, 3], 3))
print(radix_sort([170, 45, 75, 90, 802, 24, 2, 66]))
```

KẾT QUẢ KHI CHẠY

```
[1, 3, 4] [0, 2, 3, 3]
[2, 24, 45, 66, 75, 90, 170, 802]
```

- ✓ Counting sort: O(n + k), chỉ hợp khi khoảng giá trị k nhỏ.
- ✓ Radix sort: sắp theo từng chữ số, từ hàng đơn vị trở lên.

So sánh các thuật toán sắp xếp

Thuật toán	Tốt nhất	Trung bình	Xấu nhất	Bộ nhớ	Ổn định
Bubble	n	n^2	n^2	1	Có
Selection	n^2	n^2	n^2	1	Không
Insertion	n	n^2	n^2	1	Có
Merge	n log n	n log n	n log n	n	Có
Quick	n log n	n log n	n^2	log n	Không
Heap	n log n	n log n	n log n	1	Không
Counting	n + k	n + k	n + k	n + k	Có
Timsort	n	n log n	n log n	n	Có

Các ô đều là Big-O, k là khoảng giá trị.

Ổn định: hai phần tử bằng nhau giữ nguyên thứ tự ban đầu. Quan trọng khi sắp xếp theo nhiều tiêu chí.

```
people = [("An", 25), ("Bình", 20), ("Chi", 25)]
print(sorted(people, key=lambda p: p[1]))
print(sorted(people, key=lambda p: (-p[1], p[0])))
```

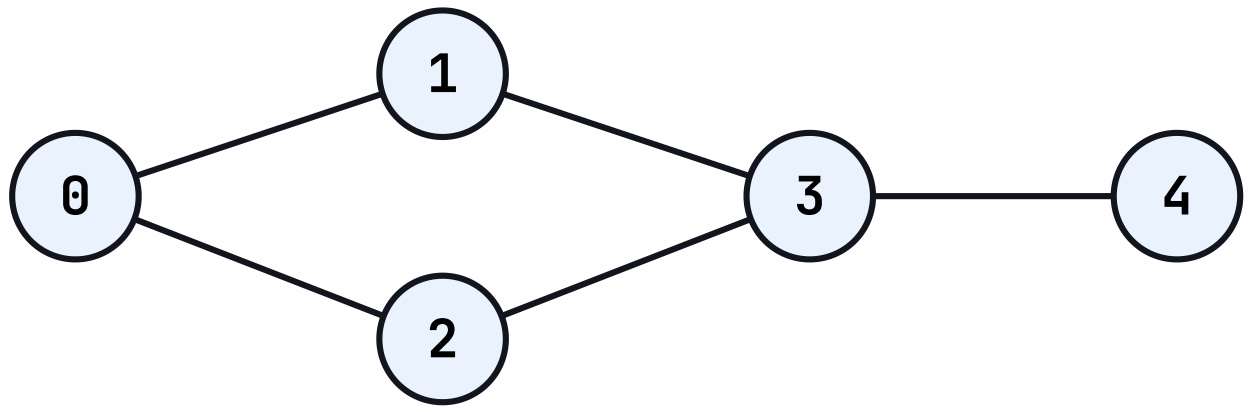
KẾT QUẢ KHI CHẠY

```
[('Bình', 20), ('An', 25), ('Chi', 25)]
[('An', 25), ('Chi', 25), ('Bình', 20)]
```

Trong thực tế, hãy dùng hàm sắp xếp có sẵn. `sorted` của Python là Timsort và ổn định.

51 Đồ thị và cách biểu diễn

Đỉnh là đối tượng, cạnh là quan hệ. Mạng xã hội, bản đồ, phụ thuộc giữa các gói đều là đồ thị.




```
edges = [(0, 1), (0, 2), (1, 3), (2, 3), (3, 4)]
n = 5

adj = [[] for _ in range(n)]          # danh sách kề
for u, v in edges:
    adj[u].append(v)
    adj[v].append(u)                  # vô hướng: thêm 2 chiều
print(adj)

mat = [[0] * n for _ in range(n)]     # ma trận kề
for u, v in edges:
    mat[u][v] = mat[v][u] = 1
print(mat[0])
```

KẾT QUẢ KHI CHẠY

```
[[1, 2], [0, 3], [0, 3], [1, 2, 4], [3]]
[0, 1, 1, 0, 0]
```

Cách lưu	Bộ nhớ	Kiểm tra cạnh	Hợp với
Danh sách kề	O(V + E)	O(bậc)	Đồ thị thưa, phổ biến nhất
Ma trận kề	O(V²)	O(1)	Đồ thị dày, V nhỏ

V là số đỉnh, E là số cạnh. DAG là đồ thị có hướng không có chu trình.

52 BFS: duyệt theo tầng

Đi từ gần tới xa bằng hàng đợi. Lần đầu chạm tới một đỉnh chính là đường đi ít cạnh nhất.

```
from collections import deque

def bfs(adj, s):
    dist = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in dist:    # đánh dấu ngay khi đưa vào
                dist[v] = dist[u] + 1
                q.append(v)
    return dist

adj = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
print(bfs(adj, 0))
```

KẾT QUẢ KHI CHẠY

```
{0: 0, 1: 1, 2: 1, 3: 2, 4: 3}
```

- Thời gian O(V + E), bộ nhớ O(V).
- Đánh dấu khi đưa vào hàng đợi, không phải khi lấy ra. Làm sai thì một đỉnh bị thêm nhiều lần.
- Cần đường đi cụ thể: lưu thêm `parent[v] = u` rồi lần ngược từ đích.
- Chỉ cho đường ngắn nhất khi mọi cạnh có cùng độ dài.

Thấy đề hỏi "ít bước nhất" trên lưới hoặc trạng thái, hãy nghĩ tới BFS.

53 Đếm đảo trên lưới

Lưới cũng là đồ thị: mỗi ô là đỉnh, ô kề nhau có cạnh nối. Không cần dựng đồ thị riêng.

```python
def islands(grid):
    R, C = len(grid), len(grid[0])
    seen = set()

    def flood(r, c):                    # loang hết một đảo
        st = [(r, c)]
        while st:
            x, y = st.pop()
            if not (0 <= x < R and 0 <= y < C):
                continue
            if grid[x][y] == "0" or (x, y) in seen:
                continue
            seen.add((x, y))
            st += [(x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)]

    count = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                flood(r, c)
    return count

print(islands(["11000", "11000", "00100", "00011"]))
```

KẾT QUẢ KHI CHẠY

```
3
```

- ✓ Thời gian O(R · C), vì mỗi ô chỉ xử lý một lần.
- ✓ Dùng stack thay đệ quy để tránh vượt giới hạn đệ quy trên lưới lớn.

54 DFS và phát hiện chu trình

Đi sâu hết một nhánh rồi quay lại. Trên đồ thị có hướng, gặp lại đỉnh đang xét nghĩa là có chu trình.

```
def dfs_order(adj, s):
    seen, order = set(), []
    def go(u):
        seen.add(u)
        order.append(u)
        for v in adj[u]:
            if v not in seen:
                go(v)
    go(s)
    return order

def has_cycle(adj):                  # đồ thị có hướng
    state = {u: 0 for u in adj}      # 0 chưa, 1 đang xét, 2 xong
    def go(u):
        state[u] = 1
        for v in adj[u]:
            if state[v] == 1 or (state[v] == 0 and go(v)):
                return True
        state[u] = 2
        return False
    return any(state[u] == 0 and go(u) for u in adj)

g = {0: [1, 2], 1: [3], 2: [3], 3: []}
print(dfs_order(g, 0), has_cycle(g))
g[3].append(0)
print(has_cycle(g))
```

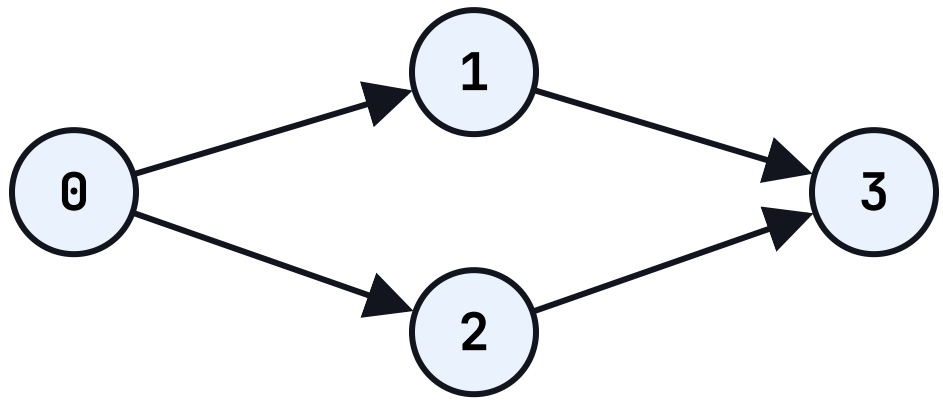
KẾT QUẢ KHI CHẠY

```
[0, 1, 3, 2] False
True
```

Đồ thị vô hướng: dùng DFS và bỏ qua cạnh về đỉnh cha, hoặc dùng Union-Find.

55 Sắp xếp topo

Xếp thứ tự công việc sao cho việc phụ thuộc luôn đứng sau. Chỉ làm được trên DAG.




```python
from collections import deque

def topo(n, edges):
    adj = [[] for _ in range(n)]
    indeg = [0] * n
    for a, b in edges:                  # a phải làm trước b
        adj[a].append(b)
        indeg[b] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else None   # None: có vòng

print(topo(4, [(0, 1), (0, 2), (1, 3), (2, 3)]))
print(topo(2, [(0, 1), (1, 0)]))
```

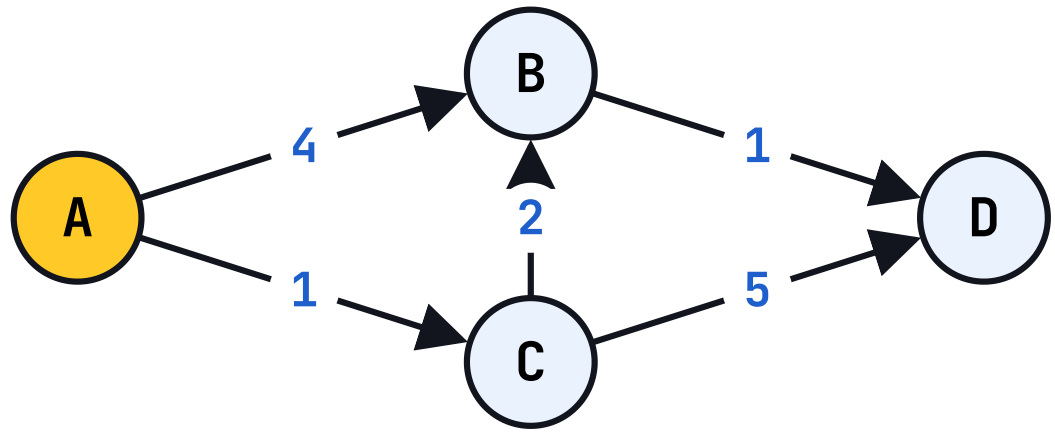
KẾT QUẢ KHI CHẠY

```
[0, 1, 2, 3]
None
```

Thuật toán Kahn, O(V + E). Dùng cho thứ tự học môn, build module, cài gói.

56 Dijkstra

Luôn mở rộng từ đỉnh đang gần nhất. Đúng khi mọi cạnh có trọng số không âm.




```python
import heapq

def dijkstra(adj, s):
    dist = {u: float("inf") for u in adj}
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:            # bản ghi cũ, bỏ qua
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist

adj = {"A": [("B", 4), ("C", 1)], "B": [("D", 1)],
       "C": [("B", 2), ("D", 5)], "D": []}
print(dijkstra(adj, "A"))
```

KẾT QUẢ KHI CHẠY

```
{'A': 0, 'B': 3, 'C': 1, 'D': 4}
```

- ✓ Dùng heap: O((V + E) log V).
- ✓ Tới B qua C tốn 3, rẻ hơn đi thẳng tốn 4.

57 Bellman-Ford

Dijkstra sai khi có cạnh âm. Bellman-Ford xử lý được, và còn phát hiện chu trình âm.

```python
def bellman_ford(n, edges, s):
    dist = [float("inf")] * n
    dist[s] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    for u, v, w in edges:                # lần thứ n vẫn giảm được
        if dist[u] + w < dist[v]:
            return None                  # có chu trình âm
    return dist

e1 = [(0, 1, 4), (0, 2, 5), (1, 2, -3), (2, 3, 2)]
e2 = [(0, 1, 1), (1, 2, -2), (2, 1, 1)]
print(bellman_ford(4, e1, 0))
print(bellman_ford(3, e2, 0))
```

KẾT QUẢ KHI CHẠY

```
[0, 4, 1, 3]
None
```

- Đường đi ngắn nhất có tối đa V − 1 cạnh, nên chỉ cần V − 1 vòng cập nhật.
- Thời gian O(V · E), chậm hơn Dijkstra.
- Chu trình âm làm đường đi nhỏ vô hạn, nên không có đáp án.

58 Floyd-Warshall

Tìm đường ngắn nhất giữa mọi cặp đỉnh chỉ bằng 3 vòng lặp. Hợp khi V nhỏ, khoảng vài trăm.

```
INF = float("inf")
d = [[0, 3, INF, 7],
     [8, 0, 2, INF],
     [5, INF, 0, 1],
     [2, INF, INF, 0]]
n = len(d)

for k in range(n):                  # k phải ở vòng ngoài cùng
    for i in range(n):
        for j in range(n):
            d[i][j] = min(d[i][j], d[i][k] + d[k][j])

for row in d:
    print(row)
```

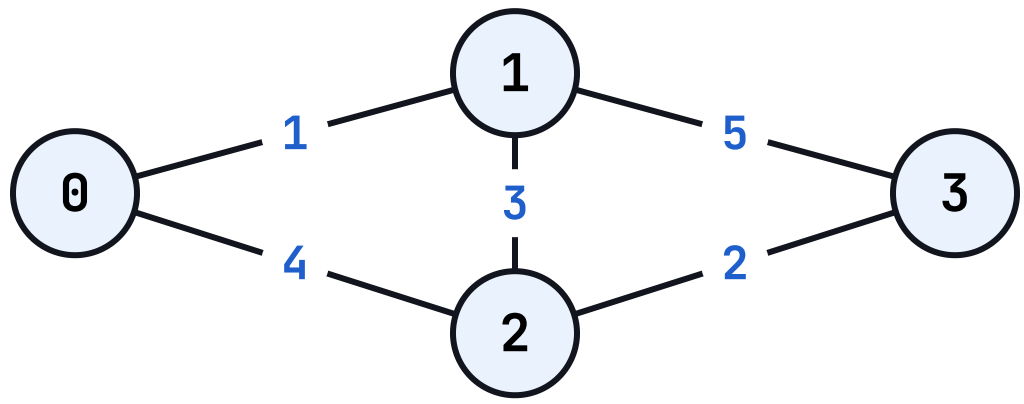
KẾT QUẢ KHI CHẠY

```
[0, 3, 5, 6]
[5, 0, 2, 3]
[3, 6, 0, 1]
[2, 5, 7, 0]
```

- Ý tưởng: lần lượt cho phép đi qua đỉnh k, xem có đường nào ngắn hơn không.
- Thời gian $O(V^3)$, bộ nhớ $O(V^2)$.
- Sau khi chạy, `d[i][i] < 0` nghĩa là có chu trình âm.

59 Cây khung nhỏ nhất: Kruskal

Nối tất cả các đỉnh với tổng chi phí nhỏ nhất. Lấy cạnh rẻ nhất trước, bỏ cạnh tạo vòng.




```python
def kruskal(n, edges):
    p = list(range(n))
    def find(x):
        while p[x] != x:
            p[x] = p[p[x]]
            x = p[x]
        return x
    total, used = 0, []
    for w, u, v in sorted(edges):      # cạnh rẻ trước
        a, b = find(u), find(v)
        if a != b:                     # không tạo vòng
            p[a] = b
            total += w
            used.append((u, v))
    return total, used

e = [(1, 0, 1), (4, 0, 2), (3, 1, 2), (2, 2, 3), (5, 1, 3)]
print(kruskal(4, e))
```

KẾT QUẢ KHI CHẠY

```
(6, [(0, 1), (2, 3), (1, 2)])
```

Thời gian O(E log E), chủ yếu do sắp xếp cạnh. Union-Find giúp kiểm tra vòng rất nhanh.

60 Cây khung nhỏ nhất: Prim

Bắt đầu từ một đỉnh, mỗi lần nối thêm đỉnh mới bằng cạnh rẻ nhất đang chạm tới cây.

```
import heapq

def prim(adj, s=0):
    seen, total = set(), 0
    pq = [(0, s)]
    while pq:
        w, u = heapq.heappop(pq)
        if u in seen:
            continue
        seen.add(u)
        total += w
        for v, c in adj[u]:
            if v not in seen:
                heapq.heappush(pq, (c, v))
    return total

adj = {0: [(1, 1), (2, 4)], 1: [(0, 1), (2, 3), (3, 5)],
       2: [(0, 4), (1, 3), (3, 2)], 3: [(1, 5), (2, 2)]}
print(prim(adj))
```

KẾT QUẢ KHI CHẠY

```
6
```

	Kruskal	Prim
Cách làm	Duyệt cạnh theo giá	Mở rộng từ một đỉnh
Cần	Union-Find	Heap
Độ phức tạp	O(E log E)	O(E log V)
Hợp với	Đồ thị thưa, có sẵn danh sách cạnh	Đồ thị dày, có danh sách kề

Chọn thuật toán đồ thị nào?

Bài toán	Dùng	Chi phí
Đường ngắn nhất, cạnh bằng nhau	**BFS**	O(V + E)
Trọng số không âm	**Dijkstra**	O((V + E) log V)
Có cạnh âm	**Bellman-Ford**	O(V · E)
Mọi cặp đỉnh, V nhỏ	**Floyd-Warshall**	$O(V^3)$
Thứ tự phụ thuộc	**Topo**	O(V + E)
Nối tất cả, chi phí nhỏ nhất	**Kruskal, Prim**	O(E log E)
Đếm thành phần liên thông	**DFS, Union-Find**	O(V + E)
Có ước lượng khoảng cách tới đích	**A***	Tùy hàm ước lượng

Câu hỏi đầu tiên luôn là: có trọng số không, có hướng không, có cạnh âm không?

61 Từ vét cạn đến tối ưu

Bắt đầu từ lời giải đơn giản nhất, rồi tìm phần việc bị lặp lại để cắt bỏ. Bài: đoạn con có tổng lớn nhất.

```python
def brute(a):                    # O(n³): thử mọi đoạn
    n = len(a)
    return max(sum(a[i:j + 1])
               for i in range(n) for j in range(i, n))

def better(a):                   # O(n²): cộng dồn, khỏi tính lại
    best = a[0]
    for i in range(len(a)):
        s = 0
        for j in range(i, len(a)):
            s += a[j]
            best = max(best, s)
    return best

def kadane(a):                   # O(n): tổng âm thì bắt đầu lại
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best

a = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(brute(a), better(a), kadane(a))
```

KẾT QUẢ KHI CHẠY

```
6 6 6
```

Trong phỏng vấn, hãy nói lời giải vét cạn trước. Nó chứng minh bạn hiểu đề và là điểm khởi đầu để tối ưu.

62 Tham lam (Greedy)

Mỗi bước chọn phương án tốt nhất lúc đó. Rất nhanh, nhưng chỉ đúng với những bài có tính chất phù hợp.

```python
def max_meetings(m):
    m.sort(key=lambda x: x[1])    # kết thúc sớm nhất trước
    count, end = 0, float("-inf")
    for s, e in m:
        if s >= end:
            count, end = count + 1, e
    return count

print(max_meetings([(1, 4), (3, 5), (0, 6), (5, 7), (8, 9)]))

def greedy_coins(x, coins):       # lấy đồng lớn nhất trước
    n = 0
    for c in sorted(coins, reverse=True):
        n += x // c
        x %= c
    return n

print(greedy_coins(6, [1, 3, 4])) # đáp án đúng là 2: 3 + 3
```

KẾT QUẢ KHI CHẠY

```
3
3
```

THAM LAM ĐÚNG
Xếp lịch họp, Huffman, Kruskal, Prim, Dijkstra.

THAM LAM SAI
Đổi tiền với mệnh giá bất kỳ, balo 0/1. Hãy dùng DP.

Trước khi dùng, hãy tìm phản ví dụ nhỏ hoặc lập luận vì sao lựa chọn đó không làm mất đáp án tối ưu.

63 Chia để trị

Chia bài toán thành các phần độc lập, giải từng phần, rồi ghép kết quả.

```
def power(a, n):                    # O(log n)
    if n == 0:
        return 1
    half = power(a, n // 2)
    return half * half * (a if n % 2 else 1)

print(power(2, 30), pow(2, 30, 1000))
```

KẾT QUẢ KHI CHẠY

```
1073741824 824
```

Công thức tổng quát: T(n) = a · T(n / b) + f(n), với a bài con cỡ n / b và chi phí ghép f(n).

Thuật toán	a	b	f(n)	Kết quả
Tìm nhị phân	1	2	O(1)	O(log n)
Merge sort	2	2	O(n)	O(n log n)
Duyệt cây cân bằng	2	2	O(1)	O(n)
Nhân Karatsuba	3	2	O(n)	O(n^1,59)

Khác DP ở chỗ các bài con không trùng nhau. Bài con trùng nhau thì nên ghi nhớ.

64 Quay lui (Backtracking)

Khung chung: chọn, đi sâu, hoàn tác. Dùng khi cần liệt kê mọi khả năng.

```python
def subsets(a):                          # O(n · 2ⁿ)
    res, cur = [], []
    def go(i):
        if i == len(a):
            res.append(cur[:])           # phải sao chép
            return
        go(i + 1)                        # không chọn a[i]
        cur.append(a[i])                 # chọn a[i]
        go(i + 1)
        cur.pop()                        # hoàn tác
    go(0)
    return res

def perms(a):                            # O(n · n!)
    res, cur = [], []
    def go():
        if len(cur) == len(a):
            res.append(cur[:])
            return
        for x in a:
            if x not in cur:             # các phần tử khác nhau
                cur.append(x)
                go()
                cur.pop()
    go()
    return res
print(subsets([1, 2, 3]))
print(perms([1, 2, 3]))
```

KẾT QUẢ KHI CHẠY

```
[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

Quên sao chép `cur[:]` là lỗi hay gặp nhất: mọi kết quả sẽ trỏ về cùng một list.

65 N-Queens

Đặt n quân hậu không ăn nhau. Cắt nhánh sớm giúp bỏ đi phần lớn không gian tìm kiếm.

```python
def queens(n):
    cols, d1, d2 = set(), set(), set()
    board, res = [], []
    def go(r):
        if r == n:
            res.append(board[:])
            return
        for c in range(n):
            if c in cols or r + c in d1 or r - c in d2:
                continue                # cắt nhánh
            cols.add(c); d1.add(r + c); d2.add(r - c)
            board.append(c)
            go(r + 1)
            board.pop()
            cols.remove(c); d1.remove(r + c); d2.remove(r - c)
    go(0)
    return res

print(len(queens(8)))
for c in queens(4)[0]:
    print("." * c + "Q" + "." * (3 - c))
```

KẾT QUẢ KHI CHẠY

```
92
.Q..
...Q
Q...
..Q.
```

- ✓ Các ô trên cùng đường chéo có chung `r + c` hoặc `r - c`.
- ✓ Dùng set để kiểm tra O(1) thay vì quét lại cả bàn cờ.

66 Quy hoạch động (DP) là gì?

Chia bài toán thành các bài con trùng nhau, và mỗi bài con chỉ giải đúng một lần.

1 Trạng thái
`dp[i]` có nghĩa là gì? Viết rõ thành một câu.

2 Công thức chuyển
`dp[i]` được tính từ những trạng thái nào?

3 Khởi tạo
Trường hợp nhỏ nhất có đáp án gì?

4 Thứ tự tính
Điền bảng theo chiều nào để ô cần dùng đã có sẵn?

DẤU HIỆU
Đề hỏi số cách, lớn nhất, nhỏ nhất, hoặc có làm được không. Lựa chọn trước ảnh hưởng lựa chọn sau.

CHI PHÍ
Thường bằng số trạng thái nhân với chi phí tính một trạng thái.

Khó nhất là định nghĩa trạng thái. Viết được câu ở bước 1 là đã đi được nửa đường.

67 Ba cách viết một bài DP

Bài leo cầu thang: n bậc, mỗi lần đi 1 hoặc 2 bậc, có bao nhiêu cách?

```
from functools import cache

@cache
def climb(n):                    # từ trên xuống: đệ quy và ghi nhớ
    return 1 if n <= 1 else climb(n - 1) + climb(n - 2)

def climb2(n):                   # từ dưới lên: điền bảng
    dp = [1, 1] + [0] * n
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def climb3(n):                   # chỉ giữ 2 biến: bộ nhớ O(1)
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b

print(climb(10), climb2(10), climb3(10))
```

KẾT QUẢ KHI CHẠY

```
89 89 89
```

Cách viết	Ưu điểm	Nhược điểm
Từ trên xuống	Viết gần công thức, chỉ tính trạng thái cần dùng	Tốn call stack
Từ dưới lên	Không đệ quy, dễ giảm bộ nhớ	Phải biết thứ tự điền

68 DP một chiều

Một chỉ số đủ mô tả trạng thái. Đây là nhóm bài DP nên luyện đầu tiên.

```python
def rob(a):                          # không lấy 2 nhà liền nhau
    take, skip = 0, 0
    for x in a:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)

def coin_change(coins, x):           # số đồng ít nhất
    dp = [0] + [float("inf")] * x
    for v in range(1, x + 1):
        for c in coins:
            if c <= v:
                dp[v] = min(dp[v], dp[v - c] + 1)
    return dp[x] if dp[x] != float("inf") else -1

def count_ways(coins, x):            # số cách đổi tiền
    dp = [1] + [0] * x
    for c in coins:                  # đồng ở ngoài: không đếm trùng
        for v in range(c, x + 1):
            dp[v] += dp[v - c]
    return dp[x]

print(rob([2, 7, 9, 3, 1]))
print(coin_change([1, 3, 4], 6), coin_change([2], 3))
print(count_ways([1, 2, 5], 5))
```

KẾT QUẢ KHI CHẠY

```
12
2 -1
4
```

Đảo hai vòng lặp trong `count_ways` thì 1 + 2 và 2 + 1 bị đếm là hai cách khác nhau.

69 Bài toán cái túi

Mỗi món chỉ lấy một lần, tổng trọng lượng không vượt W. `dp[c]` là giá trị lớn nhất với sức chứa c.

```
def knapsack(items, W):                  # items: (nặng, giá trị)
    dp = [0] * (W + 1)
    for w, v in items:
        for c in range(W, w - 1, -1):    # duyệt ngược
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[W]

print(knapsack([(1, 1), (3, 4), (4, 5), (5, 7)], 7))

def subset_sum(a, t):                    # có tập con tổng t không?
    reach = {0}
    for x in a:
        reach |= {s + x for s in reach if s + x <= t}
    return t in reach

print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

KẾT QUẢ KHI CHẠY

```
9
True
```

Biến thể	Cách duyệt c	Lý do
0/1: mỗi món một lần	Từ lớn xuống nhỏ	Không dùng lại món vừa xét
Không giới hạn số lượng	Từ nhỏ lên lớn	Được phép lấy lại

Thời gian O(n · W), bộ nhớ O(W). W rất lớn thì cách này không còn phù hợp.

70 Dãy con chung dài nhất

DP hai chiều trên hai chuỗi. Đây là nền tảng của các công cụ so sánh văn bản.

```python
def lcs(a, b):
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    res, i, j = [], n, m                    # lần ngược lấy kết quả
    while i and j:
        if a[i - 1] == b[j - 1]:
            res.append(a[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[n][m], "".join(reversed(res))

print(lcs("ABCBDAB", "BDCABA"))
```

KẾT QUẢ KHI CHẠY

```
(4, 'BCBA')
```

- ✓ `dp[i][j]`: độ dài LCS của i ký tự đầu của a và j ký tự đầu của b.
- ✓ Hai ký tự bằng nhau thì đi chéo, khác nhau thì lấy ô lớn hơn.
- ✓ Dãy con không cần liền nhau, khác với chuỗi con.
- ✓ Thời gian và bộ nhớ O(n · m).

71 Khoảng cách chỉnh sửa

Số thao tác chèn, xóa, thay ít nhất để biến chuỗi này thành chuỗi kia.

```
def edit(a, b):
    prev = list(range(len(b) + 1))
    for i, x in enumerate(a, 1):
        cur = [i]
        for j, y in enumerate(b, 1):
            cur.append(min(prev[j] + 1,          # xóa
                           cur[j - 1] + 1,       # chèn
                           prev[j - 1] + (x != y)))  # thay
        prev = cur
    return prev[-1]

print(edit("kitten", "sitting"), edit("horse", "ros"))
```

KẾT QUẢ KHI CHẠY

```
3 3
```

Từ ô nào	Nghĩa
Ô trên `prev[j]`	Xóa ký tự của a
Ô trái `cur[j - 1]`	Chèn ký tự của b
Ô chéo `prev[j - 1]`	Thay, hoặc giữ nguyên nếu bằng nhau

- ✓ Chỉ cần giữ một hàng trước đó, bộ nhớ giảm còn O(m).
- ✓ Ứng dụng: gợi ý sửa chính tả, so khớp tên gần giống.

72 Dãy con tăng dài nhất

Một bài, hai lời giải: DP O(n²) dễ hiểu, và tìm nhị phân O(n log n) nhanh hơn.

```
from bisect import bisect_left

def lis_n2(a):                      # O(n²)
    dp = [1] * len(a)
    for i in range(len(a)):
        for j in range(i):
            if a[j] < a[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp, default=0)

def lis(a):                         # O(n log n)
    tails = []                      # đuôi nhỏ nhất của dãy dài k + 1
    for x in a:
        i = bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x            # thay bằng đuôi nhỏ hơn
    return len(tails)

a = [10, 9, 2, 5, 3, 7, 101, 18]
print(lis_n2(a), lis(a))
```

KẾT QUẢ KHI CHẠY

```
4 4
```

- ✓ Đuôi càng nhỏ thì càng dễ nối thêm số sau.
- ✓ `tails` cho độ dài đúng, nhưng chưa chắc là một dãy con thật.
- ✓ Muốn tăng không ngặt thì dùng `bisect_right`.

73 Thao tác bit

Làm việc trực tiếp trên từng bit, rất nhanh và tiết kiệm bộ nhớ.

```
a, b = 0b1100, 0b1010
print(a & b, a | b, a ^ b, a << 1, a >> 1)

def is_pow2(x):
    return x > 0 and x & (x - 1) == 0

def single(nums):                # số chỉ xuất hiện một lần
    r = 0
    for x in nums:
        r ^= x                   # x ^ x = 0
    return r

print(is_pow2(64), is_pow2(65), single([4, 1, 2, 1, 2]))
print(bin(13), bin(13).count("1"), 13 & -13)

items = ["a", "b", "c"]          # mọi tập con bằng bitmask
print(["".join(x for i, x in enumerate(items) if m >> i & 1)
       for m in range(1 << 3)])
```

KẾT QUẢ KHI CHẠY

```
8 14 6 24 6
True False 4
0b1101 3 1
['', 'a', 'b', 'ab', 'c', 'ac', 'bc', 'abc']
```

Biểu thức	Tác dụng
`x & 1`	Kiểm tra số lẻ
`x & (x - 1)`	Tắt bit 1 thấp nhất
`x & -x`	Lấy bit 1 thấp nhất
`x | (1 << k)`	Bật bit thứ k

74 KMP: tìm chuỗi trong chuỗi

Khi lệch, KMP không quay lại từ đầu mà dùng bảng tiền tố để nhảy. Từ O(n · m) xuống O(n + m).

```python
def prefix_table(p):
    t, k = [0] * len(p), 0
    for i in range(1, len(p)):
        while k and p[i] != p[k]:
            k = t[k - 1]          # lùi về tiền tố ngắn hơn
        if p[i] == p[k]:
            k += 1
        t[i] = k
    return t

def kmp(s, p):
    t, k, res = prefix_table(p), 0, []
    for i, c in enumerate(s):
        while k and c != p[k]:
            k = t[k - 1]
        if c == p[k]:
            k += 1
        if k == len(p):
            res.append(i - k + 1)
            k = t[k - 1]
    return res

print(prefix_table("ababaca"))
print(kmp("abababcabab", "abab"))
```

KẾT QUẢ KHI CHẠY

```
[0, 0, 1, 2, 3, 0, 1]
[0, 2, 7]
```

- ✓ `t[i]`: độ dài tiền tố dài nhất đồng thời là hậu tố của `p[:i + 1]`.
- ✓ Cách khác: Rabin-Karp dùng hash để so sánh nhanh từng đoạn.

Nhìn đề, đoán kỹ thuật

Dấu hiệu trong đề	Nghĩ tới
Mảng đã sắp xếp, tìm cặp số	**Two Pointers**
Đoạn con liên tiếp dài nhất, ngắn nhất	**Sliding Window**
Nhiều truy vấn tổng đoạn	**Prefix Sum**
Đếm tần suất, kiểm tra đã thấy chưa	**Hash**
Phần tử lớn hơn gần nhất	**Monotonic Stack**
K lớn nhất, luồng dữ liệu liên tục	**Heap**
Giá trị nhỏ nhất sao cho điều kiện đúng	**Tìm nhị phân đáp án**
Ít bước nhất trên lưới, trạng thái	**BFS**
Phụ thuộc trước sau	**Topo**
Gộp nhóm, kiểm tra cùng nhóm	**Union-Find**
Liệt kê mọi tổ hợp, hoán vị	**Backtracking**
Số cách, tối ưu qua nhiều lựa chọn	**DP**
Tìm theo tiền tố chuỗi	**Trie**

Đọc giới hạn n trước, rồi đối chiếu bảng này. Bạn sẽ loại được phần lớn hướng đi sai.

6 bước giải bài khi phỏng vấn

1. **Hỏi rõ đề**
   Giới hạn n, kiểu dữ liệu, có trùng không, mảng rỗng thì sao?

2. **Lấy ví dụ**
   Tự chạy một ví dụ nhỏ để chắc đã hiểu đúng.

3. **Nói lời giải vét cạn**
   Nêu luôn Big-O, kể cả khi biết nó chưa đủ tốt.

4. **Tối ưu rồi mới code**
   Tìm phần việc lặp lại, nói ý tưởng và chờ người phỏng vấn đồng ý.

5. **Code rõ ràng**
   Đặt tên có nghĩa, vừa viết vừa giải thích.

6. **Tự kiểm tra**
   Chạy tay với ví dụ và trường hợp biên, rồi nêu Big-O cuối cùng.

Người phỏng vấn chấm cách bạn suy nghĩ, không chỉ đáp án cuối cùng.

Những lỗi rất hay gặp

```
a = [1, 2, 2, 3]
for x in a:
    if x == 2:
        a.remove(x)        # sửa list khi đang duyệt
print(a)                   # còn sót một số 2

def add(x, box=[]):        # mặc định dùng chung giữa các lần gọi
    box.append(x)
    return box
print(add(1), add(2))
```

KẾT QUẢ KHI CHẠY

```
[1, 2, 3]
[1, 2] [1, 2]
```

CHỈ SỐ
Sai lệch 1 ở biên, dùng `<` thay vì `<=`.

TRƯỜNG HỢP BIÊN
Quên mảng rỗng, 1 phần tử, toàn số trùng, số âm.

HÀNG ĐỢI
Dùng `list.pop(0)`, biến O(n) thành O(n²).

VÒNG LẶP ẨN
`x in list` hoặc cắt list bên trong vòng lặp.

ĐỒ THỊ
Quên đánh dấu đã thăm nên lặp vô hạn.

THAM LAM
Dùng khi chưa kiểm chứng được là đúng.

TRÀN SỐ
Java, C++ cần kiểu lớn hơn. Python thì không tràn.

SO SÁNH SỐ THỰC
Dùng `==` với số thực thay vì so với sai số nhỏ.

Bảng Big-O: cấu trúc dữ liệu

Cấu trúc	Truy cập	Tìm	Thêm	Xóa
Mảng	O(1)	O(n)	O(n)	O(n)
Mảng động, ở cuối	O(1)	O(n)	O(1)*	O(1)
Linked list	O(n)	O(n)	O(1)†	O(1)†
Stack, Queue	O(1)‡	O(n)	O(1)	O(1)
Hash table	–	O(1)*	O(1)*	O(1)*
BST cân bằng	O(log n)	O(log n)	O(log n)	O(log n)
BST bị lệch	O(n)	O(n)	O(n)	O(n)
Heap	O(1)‡	O(n)	O(log n)	O(log n)
Trie, từ dài L	O(L)	O(L)	O(L)	O(L)
Union-Find	–	≈ O(1)	≈ O(1)	–

* Trung bình hoặc khấu hao. Hash table xấu nhất là O(n). † Khi đã có sẵn con trỏ tới vị trí cần thao tác; xóa trên danh sách đơn cần nút liền trước. ‡ Chỉ phần tử ở đỉnh hoặc đầu hàng.

Không có cấu trúc nào tốt nhất. Hãy chọn theo thao tác bạn dùng nhiều nhất.

Bảng Big-O: giải thuật

Giải thuật	Thời gian	Bộ nhớ thêm
Tìm nhị phân	O(log n)	O(1)
Merge sort	O(n log n)	O(n)
Quick sort	O(n log n)*	O(log n)
BFS, DFS	O(V + E)	O(V)
Sắp xếp topo	O(V + E)	O(V)
Dijkstra với heap	O((V + E) log V)	O(V + E)
Bellman-Ford	O(V · E)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal	O(E log E)	O(V + E)
KMP	O(n + m)	O(m)
LCS, Edit Distance	O(n · m)	O(n · m)
Knapsack 0/1	O(n · W)	O(W)
LIS tối ưu	O(n log n)	O(n)

* Trung bình, xấu nhất $O(n^2)$. Chỉ cần độ dài thì LCS và Edit Distance giảm được bộ nhớ còn O(m). Bộ nhớ thêm chưa tính dữ liệu đầu vào.

Đố vui: code in ra gì?

Tự trả lời trước khi lật sang trang đáp án.

Câu 1

```
i, n = 1, 0
while i < 1000:
    i *= 2
    n += 1
print(n)
```

Câu 2

```
a = [[0] * 2] * 2
a[0][1] = 5
print(a[1][1])
```

Câu 3

```
from collections import deque
d = deque([1, 2, 3])
d.appendleft(d.pop())
print(d[0])
```

Câu 4

```
import heapq
h = [5, 1, 4]
heapq.heapify(h)
heapq.heappush(h, 0)
print(h[0])
```

Câu 5

```
s = {1, 2, 2, 3, 3, 3}
print(len(s))
```

Câu 6

```
st = []
for c in "abc":
    st.append(c)
print(st.pop() + st.pop())
```

Thêm một câu khó: vòng lặp ở câu 1 có độ phức tạp bao nhiêu nếu thay 1000 bằng n?

Đáp án đố vui

Câu	Đáp án	Giải thích
1	10	$2^{10} = 1024$ là lũy thừa đầu tiên từ 1000 trở lên
2	5	Hai hàng cùng trỏ tới một list
3	3	Lấy 3 ở cuối, đưa lên đầu
4	0	Min-heap luôn giữ số nhỏ nhất ở đầu
5	3	Set bỏ phần tử trùng
6	cb	Stack: vào sau, ra trước

Câu khó: O(log n), vì i tăng gấp đôi sau mỗi vòng.

Đúng từ 5 câu trở lên là nền tảng đã khá chắc. Sai câu nào, hãy quay lại đúng chủ đề đó.

20 câu phỏng vấn (1/2)

1. Big-O dùng để làm gì?
Mô tả chi phí tăng thế nào khi dữ liệu tăng, độc lập với máy chạy.

2. Mảng khác linked list thế nào?
Mảng truy cập O(1) nhưng chèn giữa chậm. Linked list ngược lại, tốn thêm bộ nhớ con trỏ.

3. Vì sao `append` là O(1) khấu hao?
Mảng tăng theo tỉ lệ nên việc chép dữ liệu hiếm dần, chia đều chỉ còn hằng số.

4. Stack và Queue khác nhau ra sao?
Stack vào sau ra trước. Queue vào trước ra trước.

5. Hash table xử lý va chạm thế nào?
Xâu chuỗi hoặc dò địa chỉ mở, kèm mở rộng bảng khi quá đầy.

6. Khi nào hash table thành O(n)?
Khi quá nhiều khóa rơi vào cùng một ô, do hàm băm kém hoặc bị tấn công có chủ đích.

7. BST khác heap thế nào?
BST giữ thứ tự toàn cục, tìm kiếm nhanh. Heap chỉ đảm bảo đỉnh lớn nhất hoặc nhỏ nhất.

8. Vì sao database dùng B+Tree?
Mỗi nút chứa nhiều khóa nên cây thấp, ít lần đọc đĩa, duyệt khoảng giá trị tốt.

9. BFS khác DFS thế nào?
BFS dùng hàng đợi, đi theo tầng. DFS dùng stack hoặc đệ quy, đi sâu trước.

10. Vì sao Dijkstra không dùng được với cạnh âm?
Đỉnh đã chốt vẫn có thể có đường rẻ hơn đi qua cạnh âm.

20 câu phỏng vấn (2/2)

11. Quick sort và merge sort, chọn cái nào?
Quick sort tốn ít bộ nhớ, thường nhanh trên mảng. Merge sort ổn định và luôn O(n log n).

12. Sắp xếp ổn định là gì?
Hai phần tử bằng nhau giữ nguyên thứ tự ban đầu.

13. Khi nào tìm nhị phân dùng được?
Khi điều kiện đơn điệu: từ một điểm trở đi kết quả chỉ đúng hoặc chỉ sai.

14. DP khác chia để trị thế nào?
DP có bài con trùng nhau nên phải ghi nhớ. Chia để trị có các bài con độc lập.

15. DP khác tham lam thế nào?
DP xét mọi lựa chọn qua bài con. Tham lam chỉ chọn cái tốt nhất tại mỗi bước.

16. Từ trên xuống và từ dưới lên khác gì?
Từ trên xuống dùng đệ quy và ghi nhớ. Từ dưới lên điền bảng, thường dễ giảm bộ nhớ.

17. Phát hiện vòng trong linked list thế nào?
Dùng con trỏ nhanh và chậm, nếu gặp nhau thì có vòng. Bộ nhớ thêm O(1).

18. Tìm K phần tử lớn nhất thế nào?
Giữ min-heap cỡ K, tốn O(n log K). Chỉ cần phần tử thứ K thì dùng Quickselect.

19. Vì sao đệ quy có thể gây lỗi?
Mỗi lời gọi chiếm call stack. Quá sâu sẽ tràn, khi đó chuyển sang vòng lặp với stack tự quản.

20. Thiết kế LRU Cache O(1) thế nào?
Hash table để tìm nút, danh sách liên kết đôi để đưa nút về đầu và bỏ nút cuối.

Lộ trình 8 tuần

Mỗi ngày khoảng 1 đến 2 giờ. Học hết lý thuyết của tuần, rồi làm bài theo dạng.

Tuần	Nội dung	Xem ở
1	Big-O, đệ quy, mảng, Two Pointers	Phần 1, 2
2	Sliding Window, Prefix Sum, Hash	Phần 2, 4
3	Linked List, Stack, Queue	Phần 3
4	Cây, BST, Heap	Phần 5
5	Tìm nhị phân, sắp xếp, Trie, Union-Find	Phần 5, 6
6	BFS, DFS, Topo, Dijkstra	Phần 7
7	Tham lam, quay lui, DP cơ bản	Phần 8
8	DP nâng cao, thi thử, ôn lỗi sai	Phần 8, 9

Làm ít mà nhớ lâu tốt hơn làm nhiều rồi quên. Mỗi bài sai hãy ghi lại vì sao sai.

30 bài nên luyện

Những bài kinh điển, tên theo tiếng Anh để dễ tìm trên các trang luyện code.

Chủ đề	Bài luyện
Mảng, Hash	Two Sum, Contains Duplicate, Group Anagrams, Product of Array Except Self
Con trỏ, cửa sổ	3Sum, Container With Most Water, Longest Substring Without Repeating Characters, Minimum Window Substring
Stack	Valid Parentheses, Daily Temperatures, Largest Rectangle in Histogram
Linked List	Reverse Linked List, Merge Two Sorted Lists, Linked List Cycle
Cây, Heap	Invert Binary Tree, Validate BST, Lowest Common Ancestor, Top K Frequent Elements, Find Median from Data Stream
Tìm nhị phân	Search in Rotated Sorted Array, Koko Eating Bananas
Đồ thị	Number of Islands, Course Schedule, Network Delay Time
Quay lui	Subsets, Permutations, N-Queens
DP	Climbing Stairs, Coin Change, Longest Increasing Subsequence

Mỗi bài hãy tự nêu được dạng bài, ý tưởng và Big-O trước khi xem lời giải.

Đi tiếp từ đây

1 **Mới bắt đầu**
Tự cài lại stack, queue, linked list, BST không nhìn tài liệu.

2 **Chuẩn bị đi thực tập**
Làm hết 30 bài ở trang trước, nói được Big-O từng bài.

3 **Chuẩn bị phỏng vấn**
Luyện giải bài trong 30 phút, vừa làm vừa nói thành tiếng.

4 **Muốn thi lập trình**
Đào sâu Segment Tree, DP nâng cao và đồ thị.

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Mọi đoạn code trong bộ đều đã chạy thử trên Python 3.14.